코드 설명

이력서, 자기소개, 온라인 명함, 학습 기록 대시보드와 그 빌드 스크립트를 함수·클래스 단위로 정리한 문서다. 설명은 소스 주석에서 뽑았고, 함수가 없는 파일은 파일 역할만 적었다.

프로젝트 구성

루트는 취업 지원용 포트폴리오 페이지다. 이력서, 자기소개, 명함은 각각 폴더로 나뉘고, 학습 노트와 과제 원본은 study_note_record_organization 안에 있다. 그 폴더의 파이썬 스크립트가 원본을 요약 마크다운으로 바꾸고, 대시보드가 읽는 data.js와 원본 JSON을 만든다.

- index.html, css/, js/main.js — 메인 포트폴리오
- cv/ — 이력서
- cover_letter/ — 자기소개
- online_business_card/ — 스크롤 명함
- study_note_record_organization/dashboard/ — 학습·과제 카드 화면
- portfolio_achieve/ — 프로젝트 아카이브 (PROJ_01~PROJ_21)
- study_note_record_organization/scripts/ — 요약과 정적 데이터 빌드
- study_note_record_organization/data/ — 노트, 과제, 이미지 원본

디렉터리 지도

- css/ — 메인 페이지와 공통 헤더 스타일
- js/ — 메인 페이지 내비게이션, 테마 및 리소스 보호(protect-resources.js)
- cv/css, cv/js — 이력서 스타일과 재직 기간 계산
- cover_letter/css, cover_letter/js — 자기소개 스타일과 스크롤 메뉴
- online_business_card/css, online_business_card/js — 명함 애니메이션
- study_note_record_organization/dashboard/js — 카드, 검색, 히어로. data.js와 sources는 생성물
- portfolio_achieve/ — 프로젝트 아카이브(PROJ_01~PROJ_21 메타 카드), 스타일(archive.css) 및 스크립트(site.js)
- study_note_record_organization/scripts — build_output.py, build_static_site.py, extract_sources.py
- study_note_record_organization/output — 빌드가 쓰는 요약 마크다운과 catalog.json
- study_note_record_organization/data/과제분류하기 — 과제 문서 원본. 코드 파일은 이 문서에서 빼둔다
- docs/ — 기술명세서, 코드북, 코드 설명, 소스 번들, 포트폴리오 북 빌드 스크립트

메인 사이트 및 보안 가드

index.html

포트폴리오 메인 화면. 지원 동기, 성장·학습, 역량 절과 자료 메뉴를 담고, 저장된 라이트·다크 테마를 그리기 전에 적용한다.

css/styles.css

메인 페이지의 색, 글자 크기, 섹션, 라이트·다크 테마를 정한다.

css/site-header.css

여러 페이지가 공유하는 상단 헤더, 내비게이션, 자료 드롭다운, 모바일 메뉴의 배치와 색을 정한다.

js/main.js

포트폴리오 메인 페이지의 내비게이션, 자료 드롭다운, 테마 전환을 초기화한다. DOM이 준비되면 부드러운 앵커 이동, 모바일 메뉴, 섹션 포커스를 연결한다.

prefersReducedMotion

`js/main.js:17`

```
function prefersReducedMotion()
```

사용자가 동작 줄이기를 켰는지 확인한다. 켜져 있으면 스크롤을 즉시 이동하고, 아니면 부드럽게 이동한다.

- 인자: 없음
- 반환: boolean

유사코드

```
반환 mqReduceMotion ? mqReduceMotion.matches : false
```

queryAll

`js/main.js:26`

```
function queryAll(selector)
```

문서 전체에서 선택자와 맞는 요소를 배열로 돌려준다.

- 인자: selector: string CSS 선택자
- 반환: Element[]

유사코드

```
반환 Array.prototype.slice.call(document.querySelectorAll(selector))
```

queryAllIn

`js/main.js:36`

```
function queryAllIn(root, selector)
```

지정한 요소 안에서만 선택자와 맞는 요소를 배열로 돌려준다.

- 인자: root: Element 검색을 시작할 요소, selector: string CSS 선택자
- 반환: Element[]

유사코드

```
반환 Array.prototype.slice.call(root.querySelectorAll(selector))
```

initSmoothNav

`js/main.js:44`

```
function initSmoothNav()
```

상단 메뉴의 페이지 안 링크를 부드러운 스크롤로 연결한다. 햄버거 버튼으로 모바일 메뉴를 열고, Escape와 넓은 화면에서는 메뉴를 닫는다.

- 인자: 없음
- 반환: 없음

유사코드

```
nav ← document.getElementById("site-nav")
toggle ← document.querySelector(".nav-toggle")
queryAll(".site-nav__link").forEach(function (link)
  href ← link.getAttribute("href") || ""
  만약 !href.startsWith("#") || href.length < 2) return
  link.addEventListener("click", function (e)
    target ← document.getElementById(href.slice(1))
    만약 !target) return
    e.preventDefault()
    target.scrollIntoView({ behavior: prefersReducedMotion() ? "auto" : "sm…
    closeMobileNav(nav, toggle)
    target.focus({ preventScroll: true })
만약 toggle && nav
  toggle.addEventListener("click", function ()
    open ← !nav.classList.contains("is-open")
    setMobileNav(nav, toggle, open)
document.addEventListener("keydown", function (e)
  만약 e.key === "Escape" && nav && nav.classList.contains("is-open"
    closeMobileNav(nav, toggle)
    만약 toggle) toggle.focus
window.addEventListener(
"resize",
debounce(function ()
  만약 window.innerWidth > 768 && nav && toggle
    closeMobileNav(nav, toggle)
  }, 150),
```

initResourceDropdown

`js/main.js:90`

```
function initResourceDropdown()
```

자료 메뉴 드롭다운의 열기, 항목 이동, 바깥 클릭·Escape 닫기를 연결한다. 항목이 페이지 안 링크이면 해당 구역으로 스크롤한 뒤 메뉴를 닫는다.

- 인자: 없음
- 반환: 없음

유사코드

```
nav ← document.getElementById("site-nav")
navToggle ← document.querySelector(".nav-toggle")
queryAll(".site-nav__dropdown").forEach(function (dropdown)
  dropdownToggle ← dropdown.querySelector(".site-nav__dropdown-toggle")
  만약 !dropdownToggle) return
  dropdownToggle.addEventListener("click", function (e)
    e.preventDefault()
    open ← dropdownToggle.getAttribute("aria-expanded") !== "true"
    closeResourceDropdowns(dropdown)
    setResourceDropdown(dropdown, open)
  queryAllIn(dropdown, ".site-nav__dropdown-link").forEach(function (link)
    link.addEventListener("click", function (e)
      href ← link.getAttribute("href") || ""
      만약 href.startsWith("#") && href.length > 1
        target ← document.getElementById(href.slice(1))
        만약 target
          e.preventDefault()
          target.scrollIntoView({ behavior: prefersReducedMotion() ? "auto" : "sm…
          target.focus({ preventScroll: true })
      closeResourceDropdowns()
      closeMobileNav(nav, navToggle)
document.addEventListener("click", function (e)
  target ← e.target
  만약 target instanceof Element && target.closest(".site-nav__dropdown")) ret…
  closeResourceDropdowns()
document.addEventListener("keydown", function (e)
  만약 e.key === "Escape"
    closeResourceDropdowns()
```

initThemeToggle

`js/main.js:141`

```
function initThemeToggle()
```

저장된 테마를 적용하고, 버튼과 다른 탭의 저장소 변경으로 라이트·다크를 바꾼다.

- 인자: 없음
- 반환: 없음

유사코드

```
toggle ← document.querySelector("[data-theme-toggle]")
만약 !toggle) return
storedTheme ← getStoredTheme()
theme ← storedTheme || document.documentElement.getAttribute("data-defa…
applyTheme(theme, toggle)
toggle.addEventListener("click", function ()
  current ← document.documentElement.getAttribute("data-theme") === "ligh…
  next ← current === "light" ? "dark" : "light"
  setStoredTheme(next)
  applyTheme(next, toggle)
window.addEventListener("storage", function (e)
  만약 e.key !== THEME_KEY) return
  syncedTheme ← e.newValue || document.documentElement.getAttribute("data…
  applyTheme(syncedTheme, toggle)
```

applyTheme

`js/main.js:169`

```
function applyTheme(theme, toggle)
```

html 요소와 토글 버튼에 라이트 또는 다크 테마 상태와 안내 문구를 반영한다.

- 인자: theme: "dark" | "light" | string 적용할 테마. light가 아니면 dark로 둔다., toggle: Element 테마 전환 버튼
- 반환: 없음

유사코드

```
normalizedTheme ← theme === "light" ? "light" : "dark"
label ← toggle.querySelector("[data-theme-label]")
document.documentElement.setAttribute("data-theme", normalizedTheme)
toggle.setAttribute("aria-pressed", normalizedTheme === "dark" ? "true"…
toggle.setAttribute("aria-label", normalizedTheme === "dark" ? "라이트 테마로…
만약 label
  label.textContent = normalizedTheme === "dark" ? "다크" : "라이트"
```

getStoredTheme

`js/main.js:185`

```
function getStoredTheme()
```

브라우저에 저장된 테마 값을 읽는다. 저장소를 쓸 수 없으면 null을 돌려준다.

- 인자: 없음
- 반환: string|null

유사코드

```
시도
  반환 localStorage.getItem(THEME_KEY)
  } catch (e)
  반환 null
```

setStoredTheme

`js/main.js:197`

```
function setStoredTheme(theme)
```

선택한 테마를 브라우저 저장소에 남긴다. 저장이 막히면 현재 페이지만 유지한다.

- 인자: theme: string 저장할 테마 이름
- 반환: 없음

유사코드

```
시도
  localStorage.setItem(THEME_KEY, theme)
  } catch (e)
```

scrollMobileNavToDropdown

`js/main.js:209`

```
function scrollMobileNavToDropdown(dropdown)
```

좁은 화면에서 자료 드롭다운 하단이 메뉴 패널 밖으로 나가면 #site-nav를 스크롤한다.

- 인자: dropdown: Element 드롭다운 루트
- 반환: 없음

유사코드

```
만약 !window.matchMedia("(max-width: 768px)").matches) return
nav ← document.getElementById("site-nav")
만약 !nav || !nav.classList.contains("is-open")) return
menu ← dropdown.querySelector(".site-nav__dropdown-menu")
만약 !menu) return
requestAnimationFrame(function ()
  navRect ← nav.getBoundingClientRect()
  menuRect ← menu.getBoundingClientRect()
  만약 menuRect.bottom > navRect.bottom - 8
    nav.scrollTop += menuRect.bottom - navRect.bottom + 16
```

setResourceDropdown

`js/main.js:229`

```
function setResourceDropdown(dropdown, open)
```

자료 드롭다운 하나의 열림 상태, 버튼 표시, 메뉴 숨김을 맞춘다.

- 인자: dropdown: Element 드롭다운 루트, open: boolean 열지 여부
- 반환: 없음

유사코드

```
toggle ← dropdown.querySelector(".site-nav__dropdown-toggle")
menu ← dropdown.querySelector(".site-nav__dropdown-menu")
dropdown.classList.toggle("is-open", open)
만약 toggle) toggle.setAttribute("aria-expanded", open ? "true" : "false"
만약 menu) menu.hidden = !open
만약 open) scrollMobileNavToDropdown(dropdown
```

closeResourceDropdowns

`js/main.js:242`

```
function closeResourceDropdowns(except)
```

지정한 하나를 제외한 자료 드롭다운을 모두 닫는다.

- 인자: except: Element= 닫지 않을 드롭다운
- 반환: 없음

유사코드

```
queryAll(".site-nav__dropdown").forEach(function (dropdown)
  만약 except && dropdown === except) return
  setResourceDropdown(dropdown, false)
```

closeMobileNav

`js/main.js:254`

```
function closeMobileNav(nav, toggle)
```

모바일 내비게이션을 닫고, 함께 열려 있던 자료 드롭다운도 닫는다.

- 인자: nav: HTMLElement | null 내비게이션 요소, toggle: HTMLElement | null 햄버거 버튼
- 반환: 없음

유사코드

```
setMobileNav(nav, toggle, false)
```

setMobileNav

`js/main.js:265`

```
function setMobileNav(nav, toggle, open)
```

모바일 내비게이션의 열림 클래스와 버튼 안내 문구를 설정한다. 닫을 때는 자료 드롭다운도 함께 닫는다.

- 인자: nav: HTMLElement | null 내비게이션 요소, toggle: HTMLElement | null 햄버거 버튼, open: boolean 열지 여부
- 반환: 없음

유사코드

```
만약 !nav || !toggle) return
nav.classList.toggle("is-open", open)
toggle.setAttribute("aria-expanded", open ? "true" : "false")
toggle.setAttribute("aria-label", open ? "메뉴 닫기" : "메뉴 열기")
만약 !open) closeResourceDropdowns
```

debounce

`js/main.js:280`

```
function debounce(fn, wait)
```

연속 호출을 모아 마지막 호출만 wait 밀리초 뒤에 실행하는 함수를 만든다. 창 크기 변경처럼 자주 일어나는 이벤트에서 메뉴를 한 번만 닫기 위해 쓴다.

- 인자: fn: () => void 지연 실행할 함수, wait: number 대기 시간(밀리초)
- 반환: Function

유사코드

```
t
반환 function ()
  clearTimeout(t)
  t = window.setTimeout(fn, wait)
```

initSectionTargets

`js/main.js:291`

```
function initSectionTargets()
```

id가 있는 섹션에 tabindex -1을 넣어, 메뉴 이동 뒤 키보드 포커스를 받을 수 있게 한다.

- 인자: 없음
- 반환: 없음

유사코드

```
queryAll(".section[id]").forEach(function (section)
  만약 !section.hasAttribute("tabindex"
    section.setAttribute("tabindex", "-1")
```

js/protect-resources.js

자료 복사와 다른 사이트의 끼워 넣기를 막고 안내를 보여 준다. 허용된 호스트(GitHub Pages, localhost)에서만 정상 동작하며 불법 스크랩 및 임베딩을 방지한다.

isEmbedded

`js/protect-resources.js:19`

```
function isEmbedded()
```

창이 다른 사이트의 iframe 안에 끼워져 있는지 확인한다.

- 인자: 없음
- 반환: boolean iframe 임베딩 여부

유사코드

```
시도
  반환 window.top !== window.self
  } catch (e)
  반환 true
```

allowedHost

`js/protect-resources.js:31`

```
function allowedHost()
```

현재 주소의 프로토콜과 호스트가 허용 목록에 있는지 검사한다.

- 인자: 없음
- 반환: boolean 허용 호스트 여부

유사코드

```
protocol ← window.location.protocol
만약 protocol !== "http:" && protocol !== "https:") return false
반환 !!ALLOWED_HOSTS[window.location.hostname]
```

paintBlockedPage

`js/protect-resources.js:46`

```
function paintBlockedPage()
```

비인가 호스트나 iframe일 때 전체 화면을 검은 배경으로 바꾸고 차단 안내를 표시한다.

- 인자: 없음
- 반환: 없음

유사코드

```
만약 !document.body) return
document.body.innerHTML = ""
document.body.style.margin = "0"
document.body.style.background = "#111111"
document.body.style.color = "#ffffff"
notice ← document.createElement("p")
notice.textContent = MESSAGE
notice.style.cssText = "margin:0;min-height:100vh;display:flex;align-it…
document.body.appendChild(notice)
```

isEditable

`js/protect-resources.js:108`

```
function isEditable(target)
```

이벤트 대상이 입력 요소(검색창, 텍스트에어리어 등)인지 판별한다.

- 인자: target: EventTarget 판별할 대상 요소
- 반환: boolean 편집 가능 요소 여부

유사코드

```
만약 !target || !target.closest) return false
반환 !!target.closest("input, textarea, select, [contenteditable='true']")
```

showNotice

`js/protect-resources.js:116`

```
function showNotice()
```

화면 하단에 자료 퍼가기 방지 안내 토스트 팝업을 띄운다.

- 인자: 없음
- 반환: 없음

유사코드

```
notice ← document.getElementById("resource-protect-notice")
만약 !notice
  notice = document.createElement("p")
  notice.id = "resource-protect-notice"
  notice.setAttribute("role", "status")
  notice.setAttribute("aria-live", "polite")
  notice.textContent = MESSAGE
  document.body.appendChild(notice)
notice.classList.remove("is-visible")
void notice.offsetWidth
notice.classList.add("is-visible")
window.clearTimeout(hideTimer)
hideTimer = window.setTimeout(function ()
  notice.classList.remove("is-visible")
  }, HIDE_MS)
```

block

`js/protect-resources.js:139`

```
function block(event)
```

복사, 잘라내기, 우클릭 등 금지된 이벤트의 기본 동작을 막고 안내를 띄운다.

- 인자: event: Event 가로챌 브라우저 이벤트
- 반환: 없음

유사코드

```
만약 isEditable(event.target)) return
event.preventDefault()
showNotice()
```

이력서

cv/resume.html

이력서 화면. 학력과 경력의 시작·종료 날짜를 보여주고, 기간 계산 스크립트와 공통 헤더를 불러온다.

cv/css/reset.css

이력서 페이지에서 브라우저 기본 여백과 글자 스타일을 지우는 리셋이다.

cv/css/style.css

이력서 본문의 배경, 글꼴, 경력 항목 여백과 구분선을 정한다.

cv/js/ui.js

이력서의 각 경력 항목에서 시작일·종료일을 읽어 재직 기간을 계산한다. 종료일이 today이면 오늘 날짜로 바꾸고, 연·개월을 periodResult에 괄호로 표시한다.

fillCareerPeriods

`cv/js/ui.js:5`

```
function fillCareerPeriods()
```

이력서의 각 경력 항목에서 시작일·종료일을 읽어 재직 기간을 계산한다. 종료일이 today이면 오늘 날짜로 바꾸고, 연·개월을 periodResult에 괄호로 표시한다.

- 인자: 없음
- 반환: 없음

유사코드

```
$('.article').each(function ()
  startDate ← $(this).find('.startDate').text().trim()
  만약 startDate !== ''
    startDateArr ← startDate.split('.').map(str => str.trim()).filter(Boole…
    endDate ← $(this).find('.endDate').text().trim()
    endDateArr
    만약 endDate === 'today'
      now ← new Date()
      year ← now.getFullYear()
      month ← String(now.getMonth() + 1).padStart(2, '0')
      day ← String(now.getDate()).padStart(2, '0')
      endDateArr = [year, month, day]
      $(this).find('.endDate').text(`${year}. ${month}. ${day}.`)
      아니면
      endDateArr = endDate.split('.').map(str => str.trim()).filter(Boolean)
      year ← endDateArr[0]
      month ← String(parseInt(endDateArr[1], 10)).padStart(2, '0')
      day ← String(parseInt(endDateArr[2], 10)).padStart(2, '0')
      endDateArr = [year, month, day]
      $(this).find('.endDate').text(`${year}. ${month}. ${day}.`)
    startDateCalc ← new Date(startDateArr[0], startDateArr[1], startDateArr…
    endDateCalc ← new Date(endDateArr[0], endDateArr[1], endDateArr[2])
    dateCalc ← endDateCalc.getTime() - startDateCalc.getTime()
    dateCalcResult ← dateCalc / (1000 * 60 * 60 * 24)
    dateResultText
    dateResultYear ← Math.floor(dateCalcResult / 365)
    dateResultMonth ← Math.round((dateCalcResult - (dateResultYear * 365)) …
    만약 dateResultYear === 0 && dateResultMonth === 0
      dateResultText = '-'
      아니면 만약 dateResultYear === 0
      dateResultText = `${dateResultMonth}개월`
      아니면 만약 dateResultMonth === 0
      dateResultText = `${dateResultYear}년`
      아니면
      dateResultText = `${dateResultYear}년 ${dateResultMonth}개월`
    $(this).find('.periodResult').text(`(${dateResultText})`)
```

자기소개서

cover_letter/profile.html

자기소개 본문 화면. 절마다 사이드 메뉴 활성 표시와 맨 위로 이동을 쓴다.

cover_letter/css/styles.css

자기소개 페이지의 밝은 배경, 청록 강조색, 고정 사이드바를 정한다.

cover_letter/js/script.js

자기소개서 페이지에서 현재 보고 있는 절에 맞춰 사이드 메뉴를 표시하고, 맨 위로 버튼을 연결한다.

setActiveNav

`cover_letter/js/script.js:11`

```
function setActiveNav(id)
```

data-section이 id와 같은 메뉴 링크에만 활성 클래스를 붙인다.

- 인자: id: string 현재 화면에 들어온 절의 id
- 반환: 없음

유사코드

```
document.querySelectorAll(".nav-link").forEach(function (link)
  sec ← link.getAttribute("data-section")
  만약 sec === id
    link.classList.add("is-active")
    아니면
    link.classList.remove("is-active")
```

initScrollSpy

`cover_letter/js/script.js:26`

```
function initScrollSpy()
```

본문 절이 화면의 일정 비율 이상 보이면 해당 메뉴를 활성으로 바꾼다. 관찰이 시작되기 전에는 첫 절을 활성으로 둔다.

- 인자: 없음
- 반환: 없음

유사코드

```
sections ← document.querySelectorAll(".content-section[id]")
만약 !sections.length) return
observer ← new IntersectionObserver(
function (entries)
  entries.forEach(function (entry)
    만약 entry.isIntersecting && entry.intersectionRatio >= 0.25
      setActiveNav(entry.target.id)
  root: null,
  rootMargin: "-20% 0px -55% 0px",
  threshold: [0, 0.25, 0.5, 0.75, 1]
sections.forEach(function (section)
  observer.observe(section)
first ← sections[0]
만약 first && first.id) setActiveNav(first.id
```

initToTop

cover_letter/js/script.js:57

```
function initToTop()
```

맨 위로 버튼을 누르면 페이지 최상단으로 부드럽게 스크롤한다.

- 인자: 없음
- 반환: 없음

유사코드

```
btn ← document.getElementById("toTop")
만약 !btn) return
btn.addEventListener("click", function ()
  window.scrollTo({ top: 0, behavior: "smooth" })
```

명함

online_business_card/business-card.html

온라인 명함 화면. 스크롤에 따라 명함이 뒤집히고 커진 뒤, 이어서 이력 절이 나타난다.

online_business_card/css/style.css

명함의 3D 변환, 스크롤 안내, 이력 절이 보일 때의 스타일을 정한다.

online_business_card/js/script.js

스크롤에 따라 명함을 뒤집고 확대하며, 이력서 절이 보이면 나타나게 한다.

updateAnimation

`online_business_card/js/script.js:16`

```
function updateAnimation()
```

스크롤 진행률로 명함 기울기·뒤집기·확대 transform을 다시 계산해 카드에 적용한다. 앞 절반은 뒤집기, 뒤 절반은 화면을 덮도록 확대하고, 조금 내리면 스크롤 안내를 숨긴다.

- 인자: 없음
- 반환: 없음

유사코드

```
scrollY ← window.scrollY
maxScroll ← scrollContainer.offsetHeight - window.innerHeight
progress ← Math.min(Math.max(scrollY / maxScroll, 0), 1)
rotateX ← 10
rotateY ← -15
scale ← 1
만약 progress <= 0.5
  flipProgress ← progress / 0.5
  rotateX = 10 * (1 - flipProgress)
  rotateY = -15 + (195 * flipProgress)
  scale = 1
  card.style.opacity = 1
  아니면
  scaleProgress ← (progress - 0.5) / 0.5
  rotateX = 0
  rotateY = 180
  easeScale ← scaleProgress * scaleProgress * scaleProgress
  scale = 1 + (40 * easeScale); // 화면을 덮도록 크게 확대
  card.style.opacity = 1
card.style.transform = `rotateX(${rotateX}deg) rotateY(${rotateY}deg) s…
만약 scrollHint
  scrollHint.classList.toggle('is-hidden', scrollY > 24)
ticking = false
```

학습 기록 대시보드

study_note_record_organization/dashboard/index.html

학습 기록 대시보드. 카드 목록, 검색, 요약 모달, 원본 보기를 담는다.

study_note_record_organization/dashboard/css/styles.css

학습 기록 대시보드의 카드, 검색, 모달, 히어로 갤러리 색과 배치를 정한다.

study_note_record_organization/dashboard/js/app.js

학습 기록 대시보드에서 카드 목록, 검색, 요약 모달, 원본 보기를 초기화한다.

escapeHtml

`study_note_record_organization/dashboard/js/app.js:42`

```
function escapeHtml(s)
```

사용자 문자를 HTML로 넣을 때 태그가 실행되지 않도록 이스케이프한다.

- 인자: s: string 원문
- 반환: string

유사코드

```
d ← document.createElement("div")
d.textContent = s == null ? "" : String(s)
반환 d.innerHTML
```

kindBadge

`study_note_record_organization/dashboard/js/app.js:53`

```
function kindBadge(item)
```

학습 카드의 분류 배지 HTML을 만든다. 강의·독서·잡지처럼 종류가 있을 때만 반환한다.

- 인자: item: object 카탈로그 항목
- 반환: string

유사코드

```
label ← KIND_LABEL[item.category]
만약 !label) return ""
반환 (
'<span class="badge badge--' +
escapeHtml(item.category) +
'">' +
escapeHtml(label) +
"</span>"
```

getData

`study_note_record_organization/dashboard/js/app.js:69`

```
function getData()
```

빌드가 넣어 둔 window.SITE_DATA를 돌려준다. 없으면 null이다.

- 인자: 없음
- 반환: object|null

유사코드

```
반환 window.SITE_DATA || null
```

renderIntro

`study_note_record_organization/dashboard/js/app.js:76`

```
function renderIntro()
```

카탈로그 건수로 소개 문구, 학습·과제 수, 하단 총계를 채운다.

- 인자: 없음
- 반환: 없음

유사코드

```
study ← 0
assignment ← 0
catalog.forEach(function (i)
  만약 i.type === "study") study++
  아니면 만약 i.type === "assignment"
total ← catalog.length
만약 subtitleEl
  subtitleEl.textContent =
  "독서 노트·강의·과제 기록을 요약해 둔 개인 아카이브입니다. " +
  "웹·모바일·AI 학습 흐름을 한곳에서 이어 볼 수 있도록 정리했습니다."
만약 infoCompose
  infoCompose.innerHTML =
  "<strong>학습 " +
  study +
  "건</strong> — 독서·강의·자격·도구 노트<br>" +
  "<strong>과제 " +
  assignment +
  "건</strong> — 학점은행제 제출 했던 과제"
만약 noteEl
  noteEl.textContent =
  "현재 총 " +
  total +
  "개 항목을 검색·열람할 수 있습니다. 요약 md는 " +
  "<code>python scripts/build_output.py</code>로, 정적 페이지 데이터는 " +
  "<code>python scripts/build_static_site.py</code>로 갱신합니다."
만약 footerCount
  footerCount.textContent = "총 " + total + "개 기록"
```

filteredItems

`study_note_record_organization/dashboard/js/app.js:119`

```
function filteredItems()
```

검색어가 제목, 요약, 종류, 태그에 포함된 항목만 돌려준다. 검색어가 없으면 전체다.

- 인자: 없음
- 반환: object[]

유사코드

```
q ← searchQuery.trim().toLowerCase()
반환 catalog.filter(function (item)
  만약 !q) return true
  hay ← [item.title, item.summary, item.type, KIND_LABEL[item.category] |…
  .concat(item.tags || [])
  .join(" ")
  .toLowerCase()
  반환 hay.indexOf(q) !== -1
```

renderGrid

`study_note_record_organization/dashboard/js/app.js:134`

```
function renderGrid()
```

검색 결과로 카드 그리드를 다시 그린다. 결과가 없으면 빈 상태 문구를 보여 준다.

- 인자: 없음
- 반환: 없음

유사코드

```
items ← filteredItems()
grid.innerHTML = ""
만약 items.length === 0
  empty.classList.remove("hidden")
  반환
empty.classList.add("hidden")
items.forEach(function (item)
  btn ← document.createElement("button")
  btn.type = "button"
  btn.className = "card card--" + item.type
  kind ← KIND_LABEL[item.category] || ""
  btn.setAttribute("aria-label", item.title + (kind ? " " + kind : "") + …
  btn.innerHTML =
  '<p class="card-kicker">' +
  '<span class="card-label">' +
  escapeHtml(CARD_LABEL[item.type] || "RECORD") +
  "</span>" +
  kindBadge(item) +
  "</p>" +
  '<h3 class="card__title">' +
  escapeHtml(item.title) +
  "</h3>" +
  '<p class="card__summary">' +
  escapeHtml(item.summary || "요약 없음") +
  "</p>"
  btn.addEventListener("click", function ()
    openModal(item)
  grid.appendChild(btn)
```

isAssignmentSourceKey

`study_note_record_organization/dashboard/js/app.js:177`

```
function isAssignmentSourceKey(key)
```

원본 경로가 과제 폴더인지 판별한다. 과제면 열람 전에 개인정보를 가린다.

- 인자: key: string 원본 경로
- 반환: boolean

유사코드

```
반환 key.indexOf("과제분류하기") !== -1 || key.indexOf("data_과제분류하기") !== -1
```

maskAssignmentPrivacy

`study_note_record_organization/dashboard/js/app.js:186`

```
function maskAssignmentPrivacy(text)
```

과제 원문에서 지도교수 이름, 학번, 파일명 속 학번 숫자를 가린다.

- 인자: text: string 원문
- 반환: string

유사코드

```
반환 text
.replace(/(지도교수\s*[:：]\s*)\S+(?:\s*교수님)?/g, "$1000 교수님")
.replace(/((?:학\s*번|수\s*강\s*번\s*호)\s*[:：]\s*)\d+/g, "$100000000")
.replace(/\(\d{7,8}\s+/g, "(00000000 ")
.replace(/\(\d{7,8}\)/g, "(00000000)")
.replace(/_\d{7,8}_/g, "_00000000_")
```

htmlToPlainText

`study_note_record_organization/dashboard/js/app.js:200`

```
function htmlToPlainText(html)
```

HTML 원본을 태그가 없는 화면 텍스트로 바꾼다.

- 인자: html: string HTML 문자열
- 반환: string

유사코드

```
doc ← new DOMParser().parseFromString(html, "text/html")
반환 (doc.body && doc.body.innerText) || html
```

getSourceExtraImage

`study_note_record_organization/dashboard/js/app.js:210`

```
function getSourceExtraImage(key)
```

데이터베이스 총괄과제 원본에만 ER 다이어그램 이미지를 덧붙일 정보를 돌려준다.

- 인자: key: string 원본 경로
- 반환: {src: string, alt: string, caption: string |null}

유사코드

```
만약 !key) return null
k ← key
시도
  decoded ← decodeURIComponent(key)
  만약 decoded) k = decoded
  } catch (e)
만약 k.indexOf("데이터베이스") !== -1 && k.indexOf("총괄") !== -1
  반환
    src: "assets/database-er-diagram.png",
    alt: "게임개발 DB구축 ER 다이어그램",
    caption: "ER 다이어그램 (원본 PDF 도표)",
반환 null
```

renderPlainSource

`study_note_record_organization/dashboard/js/app.js:235`

```
function renderPlainSource(text, key)
```

원문 텍스트를 모달에 넣을 HTML로 만든다. 해당하면 ER 그림을 함께 넣는다.

- 인자: text: string 원문, key: string 원본 경로
- 반환: string

유사코드

```
extra ← getSourceExtraImage(key)
html =
'<article class="md-content"><div class="source-plain">' +
'<div class="source-plain__text">' +
escapeHtml(text.replace(/\r\n/g, "\n")) +
"</div>"
만약 extra
  html +=
  '<figure class="source-plain__figure">' +
  '<img src="' +
  escapeHtml(extra.src) +
  '" alt="' +
  escapeHtml(extra.alt) +
  '" loading="lazy" decoding="async" />' +
  '<figcaption>' +
  escapeHtml(extra.caption) +
  "</figcaption></figure>"
html += "</div></article>"
반환 html
```

resolveSourceChunk

`study_note_record_organization/dashboard/js/app.js:263`

```
function resolveSourceChunk(key)
```

원본 경로에 대응하는 소스 JSON 파일 이름을 찾는다. 인코딩된 경로도 풀어 본다.

- 인자: key: string 원본 경로
- 반환: string|null

유사코드

```
만약 sourceMap[key]) return sourceMap[key]
시도
  decoded ← decodeURIComponent(key)
  만약 decoded !== key && sourceMap[decoded]) return sourceMap[decoded]
  } catch (e)
반환 null
```

siteAccessAllowed

`study_note_record_organization/dashboard/js/app.js:278`

```
function siteAccessAllowed()
```

포트폴리오 가드 또는 호스트 화이트리스트를 검사하여 정당한 사이트 접근인지 판별한다.

- 인자: 없음
- 반환: boolean 접근 허용 여부

유사코드

```
guard ← window.__portfolioGuard
만약 guard && typeof guard.allowsAccess === "function") return guard.allowsA…
protocol ← window.location.protocol
host ← window.location.hostname
known ← host === "ackdark23358.github.io" || host === "localhost" || ho…
반환 (protocol === "http:" || protocol === "https:") && known
```

fetchSourceText

`study_note_record_organization/dashboard/js/app.js:292`

```
function fetchSourceText(key)
```

소스 JSON을 가져와 원문 텍스트를 돌려준다. 한 번 받은 값은 캐시에 둔다.

- 인자: key: string 원본 경로
- 반환: Promise<string|null>

유사코드

```
만약 !siteAccessAllowed
  반환 Promise.resolve(null)
만약 sourceCache[key]
  반환 Promise.resolve(sourceCache[key])
chunk ← resolveSourceChunk(key)
만약 !chunk
  반환 Promise.resolve(null)
url ← new URL("js/sources/" + chunk, window.location.href)
만약 sourceBuildId
  url.searchParams.set("v", sourceBuildId)
반환 fetch(url.href, { cache: "no-store" })
.then(function (res)
  만약 !res.ok) throw new Error("fetch failed"
  반환 res.json()
.then(function (payload)
  text ← payload && payload.text != null ? payload.text : null
  sourceCache[key] = text
  만약 payload && payload.key && payload.key !== key
    sourceCache[payload.key] = text
  반환 text
.catch(function ()
  반환 null
```

sourceDisplayName

`study_note_record_organization/dashboard/js/app.js:331`

```
function sourceDisplayName(label, key)
```

원본 보기 제목에서 링크 문구 또는 파일 이름을 쓰고, 추출본 꼬리표를 뺀다.

- 인자: label: string 링크에 보이던 문구, key: string 원본 경로
- 반환: string

유사코드

```
반환 (
(label || key.split("/").pop() || key).replace(/\s*\(텍스트 추출본\)\s*$/, ""…
"원본"
```

openSourceViewer

`study_note_record_organization/dashboard/js/app.js:343`

```
function openSourceViewer(key, label)
```

원본 보기 오버레이를 열고 텍스트를 불러온다. HTML 과제는 평문으로, 과제 원문은 개인정보를 가린다.

- 인자: key: string 원본 경로, label: string 제목에 쓸 문구
- 반환: 없음

유사코드

```
sourceOverlay.classList.remove("hidden")
sourceOverlay.setAttribute("aria-hidden", "false")
document.body.style.overflow = "hidden"
만약 !overlay.classList.contains("hidden"
  overlay.style.pointerEvents = "none"
sourceTitle.textContent = sourceDisplayName(label, key)
sourceBody.innerHTML = '<p class="loading">원본을 불러오는 중…</p>'
fetchSourceText(key).then(function (text)
  만약 text == null
    sourceBody.innerHTML =
    '<p class="error"><code>build_static_site.py</code> 실행 후 다시 시도하세요.</p>'
    반환
  만약 /\.html?$/i.test(key
    text = htmlToPlainText(text)
  만약 isAssignmentSourceKey(key
    text = maskAssignmentPrivacy(text)
  sourceBody.innerHTML = renderPlainSource(text, key)
```

closeSourceModal

study_note_record_organization/dashboard/js/app.js:374

```
function closeSourceModal()
```

원본 보기 오버레이를 닫는다. 요약 모달이 없으면 페이지 스크롤을 다시 연다.

- 인자: 없음
- 반환: 없음

유사코드

```
sourceOverlay.classList.add("hidden")
sourceOverlay.setAttribute("aria-hidden", "true")
overlay.style.pointerEvents = ""
만약 overlay.classList.contains("hidden"
  document.body.style.overflow = ""
```

bindSourceLinks

`study_note_record_organization/dashboard/js/app.js:387`

```
function bindSourceLinks(container)
```

요약 본문 안의 원본 링크 클릭을 가로채 원본 보기를 연다.

- 인자: container: Element 요약 HTML이 들어간 요소
- 반환: 없음

유사코드

```
container.addEventListener("click", function (e)
  a ← e.target.closest("a[data-source], a.source-link")
  만약 !a) return
  e.preventDefault()
  e.stopPropagation()
  key ← a.getAttribute("data-source")
  만약 key) openSourceViewer(key, a.textContent.trim
```

openModal

`study_note_record_organization/dashboard/js/app.js:402`

```
function openModal(item)
```

카드에 해당하는 요약 HTML을 모달에 넣고 원본 링크를 연결한다. 요약이 없으면 오류 문구를 보여 준다.

- 인자: item: object 카탈로그 항목
- 반환: 없음

유사코드

```
overlay.classList.remove("hidden")
overlay.setAttribute("aria-hidden", "false")
document.body.style.overflow = "hidden"
modalMeta.innerHTML =
'<p class="card-kicker">' +
'<span class="card-label">' +
escapeHtml(CARD_LABEL[item.type] || "RECORD") +
"</span>" +
kindBadge(item) +
"</p>" +
'<h2 id="modal-title" class="modal__title">' +
escapeHtml(item.title) +
"</h2>" +
'<p class="modal__summary">' +
escapeHtml(item.summary || "") +
"</p>"
html ← articles[item.slug]
만약 !html
  modalBody.innerHTML = '<p class="error">요약본이 없습니다.</p>'
  반환
modalBody.innerHTML = '<article class="md-content">' + html + "</articl…
bindSourceLinks(modalBody)
```

closeModal

study_note_record_organization/dashboard/js/app.js:434

```
function closeModal()
```

요약 모달을 닫는다. 원본 보기가 열려 있지 않으면 페이지 스크롤을 다시 연다.

- 인자: 없음
- 반환: 없음

유사코드

```
overlay.classList.add("hidden")
overlay.setAttribute("aria-hidden", "true")
만약 sourceOverlay.classList.contains("hidden"
  document.body.style.overflow = ""
```

init

study_note_record_organization/dashboard/js/app.js:446

```
function init()
```

SITE_DATA로 카탈로그와 원본 지도를 읽고 소개와 카드 그리드를 그린다. 데이터가 없으면 빌드 스크립트를 실행하라는 안내를 보여 준다.

- 인자: 없음
- 반환: 없음

유사코드

```
만약 !siteAccessAllowed
  grid.innerHTML = '<p class="error">자료에 대해서 퍼가는 것을 방지하였습니다.</p>'
  반환
data ← getData()
만약 !data || !data.catalog
  grid.innerHTML = '<p class="error">data.js가 없습니다. build_static_site.py를…
  반환
catalog = data.catalog
articles = data.articles || {}
sourceMap = data.sourceMap || {}
sourceBuildId = data.sourceBuildId || ""
sourceCache = {}
만약 data.sources && !data.sourceMap
  sourceCache = data.sources
renderIntro()
renderGrid()
```

study_note_record_organization/dashboard/js/hero.js

학습 기록 대시보드 상단에서 이미지 두 장을 번갈아 보여 주는 히어로 갤러리를 초기화한다.

heroSrc

`study_note_record_organization/dashboard/js/hero.js:15`

```
function heroSrc(rel)
```

상대 경로의 파일 이름만 인코딩해 img src로 쓸 수 있는 주소를 만든다.

- 인자: rel: string data/image 기준 상대 경로
- 반환: string

유사코드

```
slash ← rel.lastIndexOf("/")
만약 slash === -1
  반환 encodeURI(rel)
반환 rel.slice(0, slash + 1) + encodeURIComponent(rel.slice(slash + 1))
```

getHeroImages

`study_note_record_organization/dashboard/js/hero.js:28`

```
function getHeroImages()
```

히어로에 쓸 이미지 경로 목록을 가져온다. hero-images.js의 HERO_IMAGES를 우선하고, 없으면 SITE_DATA.heroImages를 본다.

- 인자: 없음
- 반환: string[]

유사코드

```
만약 window.HERO_IMAGES && window.HERO_IMAGES.length
  반환 window.HERO_IMAGES
data ← window.SITE_DATA
만약 data && data.heroImages && data.heroImages.length
  반환 data.heroImages
반환 []
```

initHero

`study_note_record_organization/dashboard/js/hero.js:43`

```
function initHero()
```

히어로 영역을 찾고 이미지가 있으면 첫 장을 띄운 뒤, 클릭과 타이머로 다음 장으로 넘긴다. 이미지가 없으면 히어로 구역을 숨긴다.

- 인자: 없음
- 반환: 없음

유사코드

```
section ← document.getElementById("hero")
stage ← document.getElementById("hero-stage")
imgA ← document.getElementById("hero-img-a")
imgB ← document.getElementById("hero-img-b")
만약 !section || !stage || !imgA || !imgB) return
images ← getHeroImages()
만약 !images.length
  section.classList.add("hidden")
  반환
section.classList.remove("hidden")
section.removeAttribute("hidden")
current ← -1
showA ← true
busy ← false
timer ← null
* @returns {HTMLImageElement}
visibleImg 을 정의한다
* @returns {HTMLImageElement}
hiddenImg 을 정의한다
* @returns {number}
pickIndex 을 정의한다
* @param {HTMLImageElement} imgEl 대상 이미지
* @param {number} index 이미지 목록 인덱스
applyImage 을 정의한다
* @param {number} index 보여줄 이미지 인덱스
swapTo 을 정의한다
nextRandom 을 정의한다
startTimer 을 정의한다
current = pickIndex()
applyImage(imgA, current)
imgA.classList.add("is-visible")
stage.addEventListener("click", function ()
  nextRandom()
  startTimer()
startTimer()
```

visibleImg

`study_note_record_organization/dashboard/js/hero.js:68`

```
function visibleImg()
```

지금 화면에 보이는 히어로 이미지 요소를 돌려준다.

- 인자: 없음

- 반환: HTMLImageElement

유사코드

```
반환 showA ? imgA : imgB
```

hiddenImg

`study_note_record_organization/dashboard/js/hero.js:76`

```
function hiddenImg()
```

다음에 보여 주기 위해 숨겨 둔 히어로 이미지 요소를 돌려준다.

- 인자: 없음
- 반환: HTMLImageElement

유사코드

```
반환 showA ? imgB : imgA
```

pickIndex

`study_note_record_organization/dashboard/js/hero.js:84`

```
function pickIndex()
```

현재 장과 다른 이미지 인덱스를 무작위로 고른다. 한 장이면 0을 돌려준다.

- 인자: 없음
- 반환: number

유사코드

```
만약 images.length === 1) return 0
idx
do
  idx = Math.floor(Math.random() * images.length)
  } while (idx === current)
반환 idx
```

applyImage

`study_note_record_organization/dashboard/js/hero.js:98`

```
function applyImage(imgEl, index)
```

이미지 요소의 src와 대체 텍스트를 목록의 해당 장으로 바꾼다.

- 인자: imgEl: HTMLImageElement 대상 이미지, index: number 이미지 목록 인덱스
- 반환: 없음

유사코드

```
rel ← images[index]
imgEl.src = heroSrc(rel)
imgEl.alt = "학습·과제 기록 갤러리 이미지"
```

swapTo

`study_note_record_organization/dashboard/js/hero.js:109`

```
function swapTo(index)
```

숨겨 둔 이미지에 다음 장을 불러온 뒤, 로드가 끝나면 보이는 장을 교체한다. 이미 전환 중이거나 같은 장이면 아무것도 하지 않는다.

- 인자: index: number 보여줄 이미지 인덱스
- 반환: 없음

유사코드

```
만약 busy || index === current) return
busy = true
next ← hiddenImg()
prev ← visibleImg()
applyImage(next, index)
current = index
finish 을 정의한다
만약 next.complete && next.naturalWidth > 0
  finish()
  아니면
  next.onload = function ()
    next.onload = null
    finish()
  next.onerror = function ()
    next.onerror = null
    busy = false
```

finish

`study_note_record_organization/dashboard/js/hero.js:121`

```
function finish()
```

새 이미지에 표시 클래스를 옮기고 전환 중 상태를 해제한다.

- 인자: 없음
- 반환: 없음

유사코드

```
next.classList.add("is-visible")
prev.classList.remove("is-visible")
showA = next === imgA
busy = false
```

nextRandom

`study_note_record_organization/dashboard/js/hero.js:145`

```
function nextRandom()
```

현재와 다른 장을 골라 교체한다.

- 인자: 없음
- 반환: 없음

유사코드

```
swapTo(pickIndex())
```

startTimer

`study_note_record_organization/dashboard/js/hero.js:153`

```
function startTimer()
```

자동 전환 타이머를 다시 시작한다. 동작 줄이기가 켜져 있거나 이미지가 한 장이면 타이머를 두지 않는다.

- 인자: 없음
- 반환: 없음

유사코드

```
만약 timer) clearInterval(timer
reduced =
window.matchMedia &&
window.matchMedia("(prefers-reduced-motion: reduce)").matches
만약 !reduced && images.length > 1
  timer = setInterval(nextRandom, AUTO_MS)
```

프로젝트 아카이브

portfolio_achieve/index.html

파일 역할 주석이 없습니다.

portfolio_achieve/css/archive.css

코탐록 메인 페이지와 같은 색, 글자, 간격을 프로젝트 아카이브에 맞춘다.

portfolio_achieve/js/site.js

prefersReducedMotion

`portfolio_achieve/js/site.js:8`

```
function prefersReducedMotion()
```

설명이 없습니다.

- 인자: 없음
- 반환: 없음

유사코드

```
반환 mqReduceMotion ? mqReduceMotion.matches : false
```

queryAll

`portfolio_achieve/js/site.js:12`

```
function queryAll(selector)
```

설명이 없습니다.

- 인자: selector
- 반환: 없음

유사코드

```
반환 Array.prototype.slice.call(document.querySelectorAll(selector))
```

initThemeToggle

`portfolio_achieve/js/site.js:16`

```
function initThemeToggle()
```

설명이 없습니다.

- 인자: 없음
- 반환: 없음

유사코드

```
toggle ← document.querySelector("[data-theme-toggle]")
만약 !toggle) return
storedTheme ← null
시도
  storedTheme = localStorage.getItem(THEME_KEY)
  } catch (e)
  storedTheme = null
theme ← storedTheme || document.documentElement.getAttribute("data-defa…
applyTheme(theme, toggle)
toggle.addEventListener("click", function ()
  current ← document.documentElement.getAttribute("data-theme") === "ligh…
  next ← current === "light" ? "dark" : "light"
  시도
    localStorage.setItem(THEME_KEY, next)
    } catch (e)
  applyTheme(next, toggle)
```

applyTheme

`portfolio_achieve/js/site.js:42`

```
function applyTheme(theme, toggle)
```

설명이 없습니다.

- 인자: theme, toggle
- 반환: 없음

유사코드

```
normalized ← theme === "light" ? "light" : "dark"
label ← toggle.querySelector("[data-theme-label]")
document.documentElement.setAttribute("data-theme", normalized)
toggle.setAttribute("aria-pressed", normalized === "dark" ? "true" : "f…
toggle.setAttribute("aria-label", normalized === "dark" ? "라이트 테마로 전환" …
만약 label) label.textContent = normalized === "dark" ? "다크" : "라이트"
```

setMobileNav

`portfolio_achieve/js/site.js:51`

```
function setMobileNav(open)
```

설명이 없습니다.

- 인자: open
- 반환: 없음

유사코드

```
nav ← document.getElementById("site-nav")
toggle ← document.querySelector(".nav-toggle")
만약 !nav || !toggle) return
nav.classList.toggle("is-open", open)
toggle.setAttribute("aria-expanded", open ? "true" : "false")
toggle.setAttribute("aria-label", open ? "메뉴 닫기" : "메뉴 열기")
만약 !open) closeDropdowns
```

setDropdown

`portfolio_achieve/js/site.js:61`

```
function setDropdown(dropdown, open)
```

설명이 없습니다.

- 인자: dropdown, open
- 반환: 없음

유사코드

```
toggle ← dropdown.querySelector(".site-nav__dropdown-toggle")
menu ← dropdown.querySelector(".site-nav__dropdown-menu")
dropdown.classList.toggle("is-open", open)
만약 toggle) toggle.setAttribute("aria-expanded", open ? "true" : "false"
만약 menu) menu.hidden = !open
```

closeDropdowns

`portfolio_achieve/js/site.js:69`

```
function closeDropdowns(except)
```

설명이 없습니다.

- 인자: except
- 반환: 없음

유사코드

```
queryAll(".site-nav__dropdown").forEach(function (dropdown)
  만약 except && dropdown === except) return
  setDropdown(dropdown, false)
```

initNav

`portfolio_achieve/js/site.js:76`

```
function initNav()
```

설명이 없습니다.

- 인자: 없음
- 반환: 없음

유사코드

```
nav ← document.getElementById("site-nav")
toggle ← document.querySelector(".nav-toggle")
만약 toggle && nav
  toggle.addEventListener("click", function ()
    setMobileNav(!nav.classList.contains("is-open"))
queryAll(".site-nav__dropdown").forEach(function (dropdown)
  dropdownToggle ← dropdown.querySelector(".site-nav__dropdown-toggle")
  만약 !dropdownToggle) return
  dropdownToggle.addEventListener("click", function (e)
    e.preventDefault()
    open ← dropdownToggle.getAttribute("aria-expanded") !== "true"
    closeDropdowns(dropdown)
    setDropdown(dropdown, open)
queryAll('a[href^="#"]').forEach(function (link)
  link.addEventListener("click", function (e)
    id ← (link.getAttribute("href") || "").slice(1)
    target ← id ? document.getElementById(id) : null
    만약 !target) return
    e.preventDefault()
    target.scrollIntoView({ behavior: prefersReducedMotion() ? "auto" : "sm…
    만약 !target.hasAttribute("tabindex")) target.setAttribute("tabindex", "-1"
    target.focus({ preventScroll: true })
    setMobileNav(false)
    closeDropdowns()
document.addEventListener("click", function (e)
  target ← e.target
  만약 target instanceof Element && target.closest(".site-nav__dropdown")) ret…
  closeDropdowns()
document.addEventListener("keydown", function (e)
  만약 e.key !== "Escape") return
  closeDropdowns()
  만약 nav && nav.classList.contains("is-open"
    setMobileNav(false)
    만약 toggle) toggle.focus
window.addEventListener("resize", function ()
  만약 window.innerWidth > 768) setMobileNav(false
```

initSectionTargets

`portfolio_achieve/js/site.js:131`

```
function initSectionTargets()
```

설명이 없습니다.

- 인자: 없음
- 반환: 없음

유사코드

```
queryAll(".section[id], .hero[id]").forEach(function (section)
  만약 !section.hasAttribute("tabindex")) section.setAttribute("tabindex", "-1"
```

빌드 스크립트

scripts 폴더와 docs 폴더의 파이썬 스크립트가 학습 원본 요약, 대시보드 데이터 갱신, 기술문서 및 포트폴리오 북 생성을 맡는다. 아래 설명은 각 함수와 클래스 독스트링이다.

study_note_record_organization/scripts/extract_sources.py

build_output.main을 호출해 원본 추출, 요약 마크다운, 정적 사이트 데이터 갱신을 한 번에 실행한다.

study_note_record_organization/scripts/build_output.py

학습 노트와 과제 원본에서 텍스트를 뽑고, 카탈로그와 요약 마크다운을 만든다.

MLStripper

`study_note_record_organization/scripts/build_output.py:33`

```
class MLStripper(HTMLParser)
```

HTML 태그 사이의 글자만 모아 순수 텍스트로 만드는 파서.

- 인자: 없음
- 반환: 없음

유사코드

```
클래스 MLStripper
  메서드 __init__
  메서드 handle_data
  메서드 get_text
```

MLStripper.__init__

`study_note_record_organization/scripts/build_output.py:35`

```
def __init__(self)
```

태그 사이에 모을 문자열 목록을 비워 둔다.

- 인자: 없음
- 반환: 없음

유사코드

```
super().__init__()
self.parts ← []
```

MLStripper.handle_data

`study_note_record_organization/scripts/build_output.py:40`

```
def handle_data(self, data: str) -> None
```

파서가 만난 텍스트 조각을 목록에 추가한다.

- 인자: data: str
- 반환: None

유사코드

```
self.parts.append(data)
```

MLStripper.get_text

`study_note_record_organization/scripts/build_output.py:44`

```
def get_text(self) -> str
```

모은 텍스트 조각을 줄바꿈으로 이어 한 문자열로 돌려준다.

- 인자: 없음
- 반환: str

유사코드

```
반환 '\n'.join(self.parts)
```

read_text_file

`study_note_record_organization/scripts/build_output.py:49`

```
def read_text_file(path: Path) -> str
```

utf-8, utf-8-sig, cp949 순으로 텍스트 파일을 읽는다. 모두 실패하면 빈 문자열을 돌려준다.

- 인자: path: Path
- 반환: str

유사코드

```
반복 enc ← ('utf-8', 'utf-8-sig', 'cp949')
  시도
    반환 path.read_text(encoding=enc)
  실패 (UnicodeDecodeError, OSError)
    다음 반복
반환 ''
```

extract_docx

`study_note_record_organization/scripts/build_output.py:59`

```
def extract_docx(path: Path) -> str
```

워드 문서의 비어 있지 않은 문단을 줄로 이어 뽑는다. python-docx가 없으면 안내 문구를 돌려준다.

- 인자: path: Path
- 반환: str

유사코드

```
시도
  모듈을 불러온다
실패 ImportError
  반환 '[python-docx not installed]'
doc ← Document(str(path))
반환 '\n'.join((p.text for p in doc.paragraphs if p.text.strip()))
```

extract_pdf

`study_note_record_organization/scripts/build_output.py:69`

```
def extract_pdf(path: Path) -> str
```

PDF 각 페이지의 텍스트를 이어 붙인다. PyMuPDF가 없으면 안내 문구를 돌려준다.

- 인자: path: Path
- 반환: str

유사코드

```
시도
  모듈을 불러온다
실패 ImportError
  반환 '[pymupdf not installed]'
text_parts ← []
범위 fitz.open(str(path))
  반복 page ← pdf
    text_parts.append(page.get_text())
반환 '\n'.join(text_parts)
```

extract_html

`study_note_record_organization/scripts/build_output.py:82`

```
def extract_html(path: Path) -> str
```

HTML 파일에서 태그를 제거하고 본문 글자만 돌려준다. 파싱에 실패하면 원문을 그대로 둔다.

- 인자: path: Path
- 반환: str

유사코드

```
raw ← read_text_file(path)
stripper ← MLStripper()
시도
  stripper.feed(raw)
  반환 stripper.get_text()
실패 Exception
  반환 raw
```

mask_secrets

`study_note_record_organization/scripts/build_output.py:122`

```
def mask_secrets(text: str, mask: str=KEY_MASK) -> str
```

본문에서 API 키·토큰 패턴을 찾아 고정 마스크 문자열로 바꾼다.

- 인자: text: str, mask: str = KEY_MASK
- 반환: str

유사코드

```
만약 not text
  반환 text
반복 pat ← _SECRET_PATTERNS
  text ← pat.sub(mask, text)
반복 pat ← _KEY_ASSIGNMENT_PATTERNS
  text ← pat.sub(f'\\g<1>{mask}\\g<3>', text)
반환 text
```

extract_source

`study_note_record_organization/scripts/build_output.py:133`

```
def extract_source(path: Path) -> str
```

확장자에 맞춰 텍스트, 워드, PDF, HTML, VB에서 본문을 뽑고 비밀 값을 가린다.

- 인자: path: Path
- 반환: str

유사코드

```
ext ← path.suffix.lower()
raw ← ''
만약 ext == '.txt'
  raw ← read_text_file(path)
아니면
  만약 ext == '.docx'
    raw ← extract_docx(path)
  아니면
    만약 ext == '.pdf'
      raw ← extract_pdf(path)
    아니면
      만약 ext in ('.html', '.htm')
        raw ← extract_html(path)
      아니면
        만약 ext == '.vb'
          raw ← read_text_file(path)
반환 mask_secrets(raw)
```

source_href

`study_note_record_organization/scripts/build_output.py:150`

```
def source_href(path: str) -> str
```

data 기준 상대 경로를 사이트에서 여는 /data/ 주소로 바꾼다. 경로 조각은 URL 인코딩한다.

- 인자: path: str
- 반환: str

유사코드

```
rel ← path.replace('\\', '/')
만약 rel.startswith('data/')
  rel ← rel[5:]
반환 '/data/' + '/'.join((quote(seg, safe='/') for seg in rel.split('/')))
```

extracted_href

`study_note_record_organization/scripts/build_output.py:158`

```
def extracted_href(path: str) -> str
```

원본 경로를 안전한 파일 이름으로 바꿔 추출 텍스트 주소 /_extracted/...txt 를 만든다.

- 인자: path: str
- 반환: str

유사코드

```
safe ← re.sub('[^\\w\\-]', '_', path)[:120]
반환 f'/_extracted/{safe}.txt'
```

source_link_target

`study_note_record_organization/scripts/build_output.py:164`

```
def source_link_target(s: SourceRef) -> tuple[str, str]
```

원본 종류에 따라 링크에 보일 이름과 클릭 주소를 정한다. 워드와 PDF는 추출 텍스트로, 텍스트·HTML·VB는 원본 경로로 연결한다.

- 인자: s: SourceRef
- 반환: tuple[str, str]

유사코드

```
p ← ROOT / s.path
만약 p.is_dir()
  vb ← p / 'Form1.vb'
  만약 vb.is_file()
    rel ← f"{s.path.replace(chr(92), '/')}/Form1.vb"
    반환 ('Form1.vb', source_href(rel))
  반환 (os.path.basename(s.path), source_href(s.path))
name ← os.path.basename(s.path)
ext ← p.suffix.lower()
만약 ext in ('.docx', '.pdf')
  반환 (f'{name} (텍스트 추출본)', extracted_href(s.path))
만약 ext in ('.html', '.htm', '.txt', '.vb')
  반환 (name, source_href(s.path))
반환 (name, source_href(s.path))
```

is_assignment_path

`study_note_record_organization/scripts/build_output.py:192`

```
def is_assignment_path(rel_path: str) -> bool
```

경로에 과제 폴더 이름이 있으면 참을 돌려준다.

- 인자: rel_path: str
- 반환: bool

유사코드

```
반환 _ASSIGNMENT_DIR in rel_path.replace('\\', '/')
```

mask_assignment_privacy

`study_note_record_organization/scripts/build_output.py:197`

```
def mask_assignment_privacy(text: str) -> str
```

과제 본문의 지도교수 이름, 학번, 파일명 속 학번 숫자를 가린다.

- 인자: text: str
- 반환: str

유사코드

```
만약 not text
  반환 text
text ← re.sub('(지도교수\\s*[:：]\\s*)\\S+(?:\\s*교수님)?', f'\\1{PROFESSOR_MASK}', te…
text ← re.sub('((?:학\\s*번|수\\s*강\\s*번\\s*호)\\s*[:：]\\s*)\\d+', f'\\1{STUDENT_I…
text ← re.sub('\\(\\d{7,8}\\s+', f'({STUDENT_ID_MASK} ', text)
text ← re.sub('\\(\\d{7,8}\\)', f'({STUDENT_ID_MASK})', text)
text ← re.sub('_\\d{7,8}_', f'_{STUDENT_ID_MASK}_', text)
반환 text
```

cache_extract

`study_note_record_organization/scripts/build_output.py:217`

```
def cache_extract(rel_path: str, content: str) -> Path
```

뽑은 본문을 output/_extracted 아래 텍스트 파일로 저장하고 그 경로를 돌려준다.

- 인자: rel_path: str, content: str
- 반환: Path

유사코드

```
safe ← re.sub('[^\\w\\-]', '_', rel_path)[:120]
out ← EXTRACTED / f'{safe}.txt'
out.parent.mkdir(parents=True, exist_ok=True)
clean_content ← mask_secrets(content)
out.write_text(clean_content or '[empty]', encoding='utf-8')
반환 out
```

parse_unit

`study_note_record_organization/scripts/build_output.py:227`

```
def parse_unit(filename: str) -> str
```

파일 이름에서 장, 일차, 강, 주차, 서론, 부록, 코딩면허 같은 학습 단위를 찾는다. 없으면 전체다.

- 인자: filename: str
- 반환: str

유사코드

```
name ← filename.replace('.txt', '')
반복 pat ← ['(\\d+[~\\-]?\\d*)\\s*장', '(\\d+[~\\-]?\\d*)\\s*일차', '(\\d+[~\\-]?\\d*…
  m ← re.search(pat, name, re.I)
  만약 m
    반환 m.group(0)
반환 '전체'
```

sort_key_unit

`study_note_record_organization/scripts/build_output.py:244`

```
def sort_key_unit(unit: str) -> tuple
```

학습 단위를 서론, 구간, 본문, 부록 순으로 정렬하기 위한 키를 만든다.

- 인자: unit: str
- 반환: tuple

유사코드

```
만약 unit == '전체'
  반환 (0, 0)
만약 '서론' in unit or '프롤로그' in unit
  반환 (1, 0)
nums ← re.findall('\\d+', unit)
n ← int(nums[0]) if nums else 99
만약 '~' in unit or '-' in unit
  반환 (2, n)
만약 '부록' in unit
  반환 (4, n)
반환 (3, n)
```

SourceRef

`study_note_record_organization/scripts/build_output.py:260`

```
class SourceRef
```

카탈로그 항목이 가리키는 원본 파일 하나. 경로, 학습 단위, 추출한 본문을 담는다.

- 인자: 없음
- 반환: 없음

유사코드

```
클래스 SourceRef
  필드 path
  필드 unit
  필드 text
```

CatalogItem

`study_note_record_organization/scripts/build_output.py:268`

```
class CatalogItem
```

학습 또는 과제 카드 하나. 제목, 슬러그, 태그, 원본 목록을 갖고 마크다운으로 렌더된다.

- 인자: 없음
- 반환: 없음

유사코드

```
클래스 CatalogItem
  필드 id
  필드 type
  필드 category
  필드 title
  필드 slug
  필드 summary
  필드 tags
  필드 sources
  메서드 load_sources
```

CatalogItem.load_sources

`study_note_record_organization/scripts/build_output.py:279`

```
def load_sources(self) -> None
```

각 원본 파일의 본문을 읽어 text에 채우고, 가린 내용을 추출 캐시에 저장한다. 폴더면 Form1.vb를 합친다.

- 인자: 없음
- 반환: None

유사코드

```
반복 s ← self.sources
  p ← ROOT / s.path
  만약 p.is_file()
    s.text ← extract_source(p)
    만약 is_assignment_path(s.path)
      cache_extract(s.path, mask_assignment_privacy(s.text))
    아니면
      cache_extract(s.path, s.text)
  아니면
    만약 p.is_dir()
      반복 vb ← sorted(p.rglob('*.vb'))
        만약 'obj' in vb.parts or 'bin' in vb.parts
          다음 반복
        만약 vb.name in ('Form1.vb', 'Form1.Designer.vb')
          t ← extract_source(vb)
          s.text ← s.text 연산 f'\n\n--- {vb.name} ---\n{t}'
      만약 is_assignment_path(s.path)
        cache_extract(s.path, mask_assignment_privacy(s.text))
      아니면
        cache_extract(s.path, s.text)
```

normalize_study_title

`study_note_record_organization/scripts/build_output.py:631`

```
def normalize_study_title(filename: str) -> str
```

노트 파일 이름에서 장·일차 번호와 독서노트·수강노트 꼬리를 떼고 과목 이름만 남긴다.

- 인자: filename: str
- 반환: str

유사코드

```
n ← filename.replace('.txt', '').replace('.docx', '')
n ← re.sub('\\s+(\\d+[~\\-]?\\d*\\s*)?(장|일차|강|주차)\\s*(독서\\s*노트|독서노트|수강노트|영상…
n ← re.sub('\\s+(독서노트|수강노트|영상노트|독서 노트|감상노트|감상 노트)\\s*$', '', n, flags=re.I)
n ← re.sub('\\s+코딩면허.*$', '', n, flags=re.I)
n ← re.sub('\\s+노트는\\s*실습.*$', '', n, flags=re.I)
n ← re.sub('\\s+실습\\s*필기.*$', '', n, flags=re.I)
n ← re.sub('\\s+실습위주.*$', '', n, flags=re.I)
n ← re.sub('\\s+통합\\(단순합치기\\).*$', '', n)
반환 n.strip()
```

collect_study_groups

study_note_record_organization/scripts/build_output.py:649

```
def collect_study_groups() -> dict[str, list[str]]
```

data 폴더의 텍스트·워드 노트를 과목 이름으로 묶는다. hwpx는 빼고, 별칭은 같은 과목으로 합친 뒤 장·강 순으로 정렬한다.

- 인자: 없음
- 반환: dict[str, list[str]]

유사코드

```
raw ← {}
skipped_hwpx ← []
반복 f ← sorted(DATA.iterdir())
  만약 not f.is_file()
    다음 반복
  ext ← f.suffix.lower()
  만약 ext == '.hwpx'
    skipped_hwpx.append(f.name)
    다음 반복
  만약 ext not in ('.txt', '.docx')
    다음 반복
  base ← normalize_study_title(f.name)
  raw.setdefault(base, []).append(f'data/{f.name}')
merged ← {}
반복 (base, paths) ← raw.items()
  canonical ← STUDY_MERGE_ALIASES.get(base, base)
  merged.setdefault(canonical, []).extend(paths)
반복 paths ← merged.values()
  paths.sort(key=lambda p: sort_key_unit(parse_unit(os.path.basename(p))))
반환 (merged, skipped_hwpx)
```

build_study_catalog

study_note_record_organization/scripts/build_output.py:683

```
def build_study_catalog() -> list[CatalogItem]
```

과목 그룹이 기대 개수가 아니면 감사 파일을 남기고 중단한다. 맞으면 학습 CatalogItem 목록을 만든다.

- 인자: 없음

- 반환: list[CatalogItem]

유사코드

```
(groups, skipped_hwpx) ← collect_study_groups()
만약 len(groups) != EXPECTED_STUDY_GROUPS
  report ← OUT / 'study-audit.txt'
  lines ← [f'학습 그룹 수: {len(groups)} (기대: {EXPECTED_STUDY_GROUPS})', f'제외 hwpx: {l…
  반복 k ← sorted(groups)
    lines.append(f' [{len(groups[k]):2d}] {k}')
  만약 skipped_hwpx
    lines.extend(['', '=== 제외 hwpx ===', *[f' {x}' for x in skipped_hwpx]])
  report.write_text('\n'.join(lines), encoding='utf-8')
  중단 SystemExit(f'학습 과목 수가 {EXPECTED_STUDY_GROUPS}개가 아닙니다 ({len(groups)}개). …
items ← []
반복 title ← sorted(groups.keys())
  paths ← groups[title]
  meta ← STUDY_META.get(title)
  만약 meta
    (slug, summary, tags, category) ← meta
  아니면
    slug ← re.sub('[^\\w]+', '-', title.lower())[:48].strip('-') or 'study-unknown'
    summary ← f'{title} 학습 노트'
    tags ← ['study']
    category ← 'book' if 'Do it' in title else 'course'
  items.append(CatalogItem(id=f'study-{slug}', type='study', category=cat…
반환 items
```

find_assign_file

`study_note_record_organization/scripts/build_output.py:734`

```
def find_assign_file(files: list[str], keywords: list[str], used: set[str], exclude_contains:
list[str] | None=None) -> str | None
```

아직 쓰지 않은 과제 파일 중 키워드가 들어 있고 제외어가 없는 첫 파일을 찾는다.

- 인자: files: list[str], keywords: list[str], used: set[str], exclude_contains: list[str] | None = None
- 반환: str | None

유사코드

```
excludes ← exclude_contains or []
반복 kw ← keywords
  반복 f ← files
    만약 f in used
      다음 반복
    만약 any((ex in f for ex in excludes))
      다음 반복
    만약 kw in f
      반환 f
반환 None
```

build_assignment_catalog

`study_note_record_organization/scripts/build_output.py:748`

```
def build_assignment_catalog() -> list[CatalogItem]
```

과제 폴더의 파일 이름과 과목 키워드를 맞춰 과제 카탈로그를 만든다. 윈도우 프로그래밍1은 텍스트 문서가 없어 출처 목록을 비워 둔다.

- 인자: 없음
- 반환: list[CatalogItem]

유사코드

```
files ← [f for f in os.listdir(ASSIGN) if (ASSIGN / f).is_file()]
used ← set()
items ← []
add 을 정의한다
add('c-language-1', 'C언어1', ['C언어1'], ['c', 'pointer'])
add('pc-skills-1', 'PC활용1', ['PC활용1'], ['office', 'pc'])
add('database', '데이터베이스', ['데이터베이스'], ['database', 'sql'])
add('multimedia-intro', '멀티미디어 개론', ['멀티미디어 개론', '멀티미디어개론'], ['multimed…
add('multimedia-telecom', '멀티미디어 통신', ['멀티미디어통신'], ['multimedia', 'tele…
add('software-engineering', '소프트웨어공학', ['소프트웨어공학'], ['software-engineer…
add('system-analysis-design', '시스템분석설계', ['시스템분석설계'], ['system', 'analy…
add('discrete-math', '이산수학', ['이산수학'], ['math'])
add('internet-marketing', '인터넷 마케팅', ['인터넷 마케팅'], ['marketing', 'intern…
add('internet-programming', '인터넷프로그래밍', ['인터넷프로그래밍_'], ['html', 'web'],…
add('internet-programming-2', '인터넷프로그래밍2', ['인터넷프로그래밍2'], ['html', 'web…
add('data-structures', '자료구조', ['자료구조'], ['algorithm', 'data-structure'…
add('information-processing', '정보처리', ['정보처리'], ['information'])
add('computer-intro', '컴퓨터개론', ['컴퓨터개론'], ['computer-science'])
add('computer-graphics-1', '컴퓨터그래픽1', ['컴퓨터그래픽1'], ['graphics'])
add('computer-network', '컴퓨터네트워크', ['컴퓨터네트워크'], ['network', 'protocol'])
items.append(CatalogItem(id='assignment-windows-programming-1', type='a…
add('windows-programming', '윈도우프로그래밍2', ['윈도우프로그래밍2'], ['document'])
반환 items
```

build_assignment_catalog.add

`study_note_record_organization/scripts/build_output.py:757`

```
def add(slug: str, title: str, keywords: list[str], tags: list[str], exclude_contains:
list[str] | None=None, extra_sources: list[SourceRef] | None=None)
```

키워드로 과제 파일을 하나 묶어 과제 CatalogItem을 목록에 추가한다. 추가 출처가 있으면 같이 넣는다.

- 인자: slug: str, title: str, keywords: list[str], tags: list[str], exclude_contains: list[str] | None = None, extra_sources: list[SourceRef] | None = None
- 반환: 없음

유사코드

```
matched ← find_assign_file(files, keywords, used, exclude_contains)
srcs ← []
만약 matched
  used.add(matched)
  srcs.append(SourceRef(path=f'data/과제분류하기/{matched}', unit='총괄과제'))
만약 extra_sources
  srcs.extend(extra_sources)
items.append(CatalogItem(id=f'assignment-{slug}', type='assignment', ca…
```

extract_meta_assignment

`study_note_record_organization/scripts/build_output.py:812`

```
def extract_meta_assignment(text: str) -> dict[str, str]
```

과제 본문에서 학번, 수강번호, 이름, 지도교수, 제출일 값을 정규식으로 골라 딕셔너리로 돌려준다.

- 인자: text: str
- 반환: dict[str, str]

유사코드

```
meta ← {}
patterns ← [('학번', '학\\s*번\\s*[:：]\\s*(\\d+)'), ('수강번호', '수\\s*강\\s*번\\s*호\\s*[:：]…
반복 (key, pat) ← patterns
  m ← re.search(pat, text)
  만약 m
    meta[key] ← m.group(1)
반환 meta
```

_assignment_line_inner

`study_note_record_organization/scripts/build_output.py:850`

```
def _assignment_line_inner(line: str) -> str
```

줄 앞의 글머리표와 중복 공백을 제거한 속 내용을 돌려준다.

- 인자: line: str
- 반환: str

유사코드

```
반환 re.sub('\\s+', ' ', line.strip().lstrip('•-*· ').strip())
```

_is_assignment_meta_line

study_note_record_organization/scripts/build_output.py:855

```
def _is_assignment_meta_line(line: str) -> bool
```

학번, 제출일, 과목명, 학과 줄처럼 과제 표지 메타인지 판별한다.

- 인자: line: str
- 반환: bool

유사코드

```
s ← line.strip()
만약 not s
  반환 False
inner ← _assignment_line_inner(s)
만약 _ASSIGNMENT_META_LINE.match(s) or _ASSIGNMENT_META_LINE.match(inner)
  반환 True
만약 _ASSIGNMENT_META_FIELDS.search(inner)
  반환 True
만약 re.match('^총\\s*괄\\s*과\\s*제\\s*$', inner, re.I)
  반환 True
만약 _ASSIGNMENT_DEPT_LINE.match(s) and re.search('[가-힣]', s)
  반환 True
반환 False
```

strip_assignment_cover_block

study_note_record_organization/scripts/build_output.py:872

```
def strip_assignment_cover_block(text: str) -> str
```

표지·과제주제 라벨·학번/제출일 등 메타 구간을 본문(서론·목차 등) 전까지 제거.

- 인자: text: str
- 반환: str

유사코드

```
lines ← text.splitlines()
kept ← []
in_cover ← True
skip_topic_lines ← False
반복 ln ← lines
  s ← ln.strip()
  만약 in_cover
    만약 not s
      다음 반복
    만약 re.fullmatch('={3,}', s)
      다음 반복
    만약 _is_assignment_meta_line(ln)
      skip_topic_lines ← False
      다음 반복
    만약 re.search('과\\s*제\\s*주\\s*제\\s*[:：]', s, re.I)
      skip_topic_lines ← True
      다음 반복
    만약 skip_topic_lines
      만약 re.match('^[•\\-\\*·]', s)
        만약 _is_assignment_meta_line(ln)
          다음 반복
        skip_topic_lines ← False
        다음 반복
      다음 반복
    만약 re.match('^(?:서론|목차|본론|결론)\\b', s)
      in_cover ← False
      kept.append(ln)
      다음 반복
    만약 s.startswith(' ') or len(s) > 55
      in_cover ← False
      kept.append(ln)
      다음 반복
    만약 len(s) <= 40
      다음 반복
    in_cover ← False
    kept.append(ln)
  아니면
    kept.append(ln)
반환 '\n'.join(kept)
```

strip_assignment_meta_text

`study_note_record_organization/scripts/build_output.py:916`

```
def strip_assignment_meta_text(text: str) -> str
```

과제 원본에서 학번·제출일·과제주제 표지 등 메타 블록을 제거.

- 인자: text: str
- 반환: str

유사코드

```
만약 not text
  반환 text
text ← re.sub('^[ \\t]*={3,}[ \\t]*\\n(?:.*?\\n)*?[ \\t]*={3,}[ \\t]*\\n+', ''…
text ← re.sub('^[ \\t]*[•\\-\\*·][ \\t]*(?:과목\\s*명|지\\s*도\\s*교\\s*수|담\\s*당\\s*…
text ← strip_assignment_cover_block(text)
kept ← []
반복 ln ← text.splitlines()
  s ← ln.strip()
  만약 not s
    kept.append('')
    다음 반복
  만약 re.fullmatch('={3,}', s)
    다음 반복
  만약 _is_assignment_meta_line(ln)
    다음 반복
  만약 _ASSIGNMENT_SECTION_HEADER.match(s)
    다음 반복
  kept.append(ln)
반환 '\n'.join(kept)
```

is_assignment_meta_bullet

`study_note_record_organization/scripts/build_output.py:952`

```
def is_assignment_meta_bullet(bullet: str) -> bool
```

요약 목록에 넣으면 안 되는 표지 정보, 경로, 코드 잡음인지 판별한다.

- 인자: bullet: str
- 반환: bool

유사코드

```
s ← bullet.strip()
만약 not s
  반환 True
inner ← _assignment_line_inner(s)
만약 _ASSIGNMENT_META_FIELDS.search(inner)
  반환 True
만약 re.match('^총\\s*괄\\s*과\\s*제\\s*$', inner, re.I)
  반환 True
만약 _ASSIGNMENT_DEPT_LINE.match(s) and re.search('[가-힣]', s)
  반환 True
만약 re.match('^C:[/\\\\]', s)
  반환 True
만약 re.match('^(?:File |Traceback|using |import |#include|Private Sub )', s…
  반환 True
반환 False
```

summarize_assignment_bullets

`study_note_record_organization/scripts/build_output.py:971`

```
def summarize_assignment_bullets(text: str, max_items: int=10) -> list[str]
```

과제 본문에서 표지 메타를 뺀 뒤, 요약에 쓸 불릿을 최대 개수만큼 고른다.

- 인자: text: str, max_items: int = 10
- 반환: list[str]

유사코드

```
cleaned ← strip_assignment_meta_text(mask_assignment_privacy(text))
raw ← summarize_bullets(cleaned, max_items * 2)
out ← []
반복 b ← raw
  만약 is_assignment_meta_bullet(b)
    다음 반복
  out.append(b)
  만약 len(out) >= max_items
    반복 종료
반환 out
```

_clip_prose

`study_note_record_organization/scripts/build_output.py:985`

```
def _clip_prose(s: str, max_len: int=200) -> str
```

문장을 최대 길이로 자르고 말끝을 부드럽게 다듬는다.

- 인자: s: str, max_len: int = 200
- 반환: str

유사코드

```
s ← re.sub('\\s+', ' ', s.strip())
s ← re.sub('^[•\\-\\*·]\\s*', '', s)
만약 not s
  반환 ''
만약 len(s) <= max_len
  반환 soften_sentence(s)
clipped ← s[:max_len - 1].rsplit(' ', 1)[0]
반환 soften_sentence(clipped + '…')
```

_is_readable_assignment_prose

`study_note_record_organization/scripts/build_output.py:997`

```
def _is_readable_assignment_prose(line: str) -> bool
```

과제 줄이 사람이 읽을 산문인지 본다. 공백 없는 긴 토큰은 제외한다.

- 인자: line: str
- 반환: bool

유사코드

```
s ← line.strip()
만약 not is_prose_line(s)
  반환 False
만약 len(s) > 48 and ' ' not in s
  반환 False
반환 True
```

extract_assignment_points

`study_note_record_organization/scripts/build_output.py:1007`

```
def extract_assignment_points(text: str, max_items: int=15) -> list[str]
```

과제 본문에서 서로 다른 읽을 수 있는 줄을 앞에서부터 최대 개수만큼 모은다.

- 인자: text: str, max_items: int = 15
- 반환: list[str]

유사코드

```
points ← []
seen ← set()
반복 ln ← text.splitlines()
  만약 not ln.strip()
    다음 반복
  ln ← re.sub('\\s+', ' ', ln.strip())
  만약 not _is_readable_assignment_prose(ln)
    다음 반복
  key ← ln[:50]
  만약 key in seen
    다음 반복
  seen.add(key)
  points.append(ln)
  만약 len(points) >= max_items
    반복 종료
반환 points
```

_split_assignment_sections

`study_note_record_organization/scripts/build_output.py:1027`

```
def _split_assignment_sections(text: str) -> dict[str, str]
```

서론, 본론, 결론 제목을 기준으로 과제 본문을 세 구간 문자열로 나눈다.

- 인자: text: str
- 반환: dict[str, str]

유사코드

```
buffers ← {'intro': [], 'body': [], 'outro': []}
active ← 'intro'
반복 ln ← text.splitlines()
  s ← ln.strip()
  만약 re.match('^서론\\s*$', s)
    active ← 'intro'
    다음 반복
  만약 re.match('^본론\\s*$', s)
    active ← 'body'
    다음 반복
  만약 re.match('^결론\\s*$', s)
    active ← 'outro'
    다음 반복
  buffers[active].append(ln)
반환 {k: '\n'.join(v) for k, v in buffers.items() if any((x.strip() for x in…
```

_has_assignment_section_markers

`study_note_record_organization/scripts/build_output.py:1046`

```
def _has_assignment_section_markers(text: str) -> bool
```

본문에 서론과 본론 제목이 둘 다 있는지 확인한다.

- 인자: text: str
- 반환: bool

유사코드

```
반환 bool(re.search('^서론\\s*$', text, re.M)) and bool(re.search('^본론\\s*$', …
```

_extract_assignment_topics

`study_note_record_organization/scripts/build_output.py:1053`

```
def _extract_assignment_topics(text: str) -> list[str]
```

목차나 짧은 제목 줄에서 과제 주제 후보를 최대 여섯 개 고른다.

- 인자: text: str
- 반환: list[str]

유사코드

```
topics ← []
prefer ← False
반복 ln ← text.splitlines()
  s ← ln.strip()
  만약 re.match('^목차\\s*$', s)
    prefer ← True
    다음 반복
  만약 not s or len(s) < 6 or len(s) > 48
    다음 반복
  만약 _is_assignment_meta_line(ln)
    다음 반복
  만약 re.match('^\\d+$', s)
    다음 반복
  만약 re.match('^(?:서론|본론|결론|목차|참고|출력)', s)
    다음 반복
  만약 not re.search('[가-힣]', s)
    다음 반복
  만약 s in topics
    다음 반복
  만약 prefer or len(topics) < 4
    topics.append(s)
  만약 len(topics) >= 6
    반복 종료
만약 len(topics) < 3
  topics ← []
  반복 ln ← text.splitlines()
    s ← ln.strip()
    만약 not s or len(s) < 8 or len(s) > 48
      다음 반복
    만약 _is_assignment_meta_line(ln) or re.match('^\\d+$', s)
      다음 반복
    만약 re.match('^(?:서론|본론|결론|목차|참고)', s)
      다음 반복
    만약 re.search('[가-힣]', s) and s not in topics
      topics.append(s)
    만약 len(topics) >= 6
      반복 종료
반환 topics
```

_assignment_summary_filler

study_note_record_organization/scripts/build_output.py:1095

```
def _assignment_summary_filler(item: CatalogItem, index: int) -> str
```

요약 문장이 부족할 때 순번에 맞춰 넣는 기본 문장을 돌려준다.

- 인자: item: CatalogItem, index: int
- 반환: str

유사코드

```
fillers ← [f'{item.title} 총괄과제에서 요구한 주제랑 형식에 맞춰 답안·산출물을 구성했어요.', '본문·
코드·도표·발표 자료 …
반환 fillers[min(index, len(fillers) - 1)]
```

_sentence_from_section

`study_note_record_organization/scripts/build_output.py:1113`

```
def _sentence_from_section(points: list[str], item: CatalogItem, section_name: str) -> str
```

서론·본론·결론 구간의 첫 문장을 그 구간에 맞는 말투의 한 문장으로 만든다.

- 인자: points: list[str], item: CatalogItem, section_name: str
- 반환: str

유사코드

```
verb ← _SECTION_VERBS.get(section_name, '정리했어요')
만약 not points
  반환 f"{section_name}에서는 {item.summary.rstrip('.')} 관련 내용을 {verb}"
core ← _clip_prose(points[0], 52).rstrip('.')
반환 f'{section_name}에서는 {core}를 중심으로 {verb}'
```

_compose_summary_from_topics

`study_note_record_organization/scripts/build_output.py:1122`

```
def _compose_summary_from_topics(item: CatalogItem, topics: list[str]) -> list[str]
```

주제 세 개로 개요, 본문, 마무리 형식의 요약 문장 세 개를 만든다.

- 인자: item: CatalogItem, topics: list[str]
- 반환: list[str]

유사코드

```
picks ← topics[:3]
동안 len(picks) < 3
  picks.append(item.summary)
(t0, t1, t2) ← (picks[0], picks[1], picks[2])
반환 [f'과제는 「{t0}」 주제로 개요와 배경부터 소개했어요.', f'「{t1}」 파트에서 개념·사례·조사 자료를
표와 도표로 정…
```

_sentence_from_points

`study_note_record_organization/scripts/build_output.py:1135`

```
def _sentence_from_points(points: list[str], opener: str, fallback: str='') -> str
```

고른 문장의 앞부분을 시작 말과 이어 요약 한 문장으로 만든다.

- 인자: points: list[str], opener: str, fallback: str = ''
- 반환: str

유사코드

```
만약 not points
  base ← fallback or '과제 원문에서 핵심만 골라'
  반환 f"{opener.rstrip('.')} {base} 정리했어요."
core ← _clip_prose(points[0], 52).rstrip('.')
반환 f"{opener.rstrip('.')} {core}를 다루며 정리했어요."
```

build_assignment_content_summary

`study_note_record_organization/scripts/build_output.py:1144`

```
def build_assignment_content_summary(item: CatalogItem, all_text: str) -> list[str]
```

과제 본문에서 구어체 한 문장 요약 3개 생성.

- 인자: item: CatalogItem, all_text: str
- 반환: list[str]

유사코드

```
cleaned ← strip_assignment_meta_text(mask_assignment_privacy(all_text))
sections ← _split_assignment_sections(cleaned)
section_keys ← [('intro', '서론'), ('body', '본론'), ('outro', '결론')]
section_lines ← []
만약 _has_assignment_section_markers(cleaned)
  반복 (key, name) ← section_keys
    sec ← sections.get(key, '').strip()
    만약 not sec
      다음 반복
    pts ← extract_assignment_points(sec, 6)
    line ← _sentence_from_section(pts, item, name)
    만약 line
      section_lines.append(line)
  만약 len(section_lines) >= 2
    만약 len(section_lines) < 3
      tail ← sections.get('outro', '') or sections.get('body', '')
      pts ← extract_assignment_points(tail, 4)
      만약 pts
        section_lines.append(_sentence_from_section([pts[-1]], item, '마무리'))
    동안 len(section_lines) < 3
      section_lines.append(_assignment_summary_filler(item, len(section_lines…
    반환 section_lines[:3]
topics ← _extract_assignment_topics(cleaned)
is_slide_pdf ← any((s.path.lower().endswith('.pdf') for s in item.sources))
만약 len(topics) >= 3 and (is_slide_pdf or len(extract_assignment_points(cle…
  반환 _compose_summary_from_topics(item, topics)
points ← extract_assignment_points(cleaned, 15)
만약 len(points) < 3
  만약 len(topics) >= 3
    반환 _compose_summary_from_topics(item, topics)
  만약 points
    openers ← ['먼저', '이어서', '마무리로는']
    lines ← [_sentence_from_points([p], openers[i], f'{item.title} 과제에서') for i, p …
    동안 len(lines) < 3
      lines.append(_assignment_summary_filler(item, len(lines)))
    반환 lines[:3]
  반환 [_assignment_summary_filler(item, 0), _assignment_summary_filler(item, …
third ← max(1, len(points) // 3)
chunks ← [points[:third], points[third:third * 2], points[third * 2:]]
openers ← ['앞부분에서는', '중간에서는', '마무리에서는']
…
```

is_prose_line

`study_note_record_organization/scripts/build_output.py:1222`

```
def is_prose_line(line: str) -> bool
```

한글이 있고 코드·로그·너무 짧거나 긴 줄이 아닌 서술 줄인지 판별한다.

- 인자: line: str
- 반환: bool

유사코드

```
s ← line.strip()
만약 len(s) < 12 or len(s) > 280
  반환 False
만약 not _HANGUL.search(s)
  반환 False
만약 _CODE_NOISE.match(s)
  반환 False
만약 s.count('=') > 4 or s.startswith('---')
  반환 False
ascii_ratio ← sum((1 for c in s if ord(c) < 128)) / len(s)
만약 ascii_ratio > 0.65 and '파이썬' not in s and ('프로그래' not in s)
  반환 False
반환 True
```

extract_study_points

`study_note_record_organization/scripts/build_output.py:1239`

```
def extract_study_points(text: str, max_items: int=8) -> list[str]
```

노트에서 서술에 쓸 만한 한국어 문장만 추출.

- 인자: text: str, max_items: int = 8
- 반환: list[str]

유사코드

```
lines ← [ln.strip() for ln in text.splitlines() if ln.strip()]
points ← []
seen ← set()
반복 ln ← lines
  만약 not is_prose_line(ln)
    다음 반복
  ln ← re.sub('\\s+', ' ', ln)
  key ← ln[:50]
  만약 key in seen
    다음 반복
  seen.add(key)
  points.append(ln)
  만약 len(points) >= max_items
    반복 종료
반환 points
```

soften_sentence

`study_note_record_organization/scripts/build_output.py:1258`

```
def soften_sentence(s: str) -> str
```

문장 끝의 불필요한 마침표와 '등의'를 정리하고, 끝이 없으면 마침표를 붙인다.

- 인자: s: str
- 반환: str

유사코드

```
s ← s.rstrip('.')
s ← re.sub('\\s*등의\\s*$', '', s)
만약 s and s[-1] not in '.!?'
  s ← s 연산 '.'
반환 s
```

weave_narrative

`study_note_record_organization/scripts/build_output.py:1267`

```
def weave_narrative(opener: str, points: list[str], fallback: str='') -> str
```

고른 문장을 구어체 서술 한 덩어리로 이어 붙인다. 쓸 문장이 없으면 fallback으로 채운다.

- 인자: opener: str, points: list[str], fallback: str = ''

- 반환: str

유사코드

```
cleaned ← [soften_sentence(p).rstrip('.') for p in points if p.strip()]
만약 not cleaned
  base ← fallback or '교재·강의 흐름을 따라가며 전체 개념을 익히는 데 집중'
  반환 f"{opener.rstrip('.')} {base}했어요."
만약 len(cleaned) == 1
  반환 f"{opener.rstrip('.')} {cleaned[0]} 내용을 중심으로 공부했어요."
first ← ' '.join(cleaned[:3])
rest ← ' '.join(cleaned[3:6])
만약 rest
  반환 f"{opener.rstrip('.')} {first} 같은 내용을 먼저 잡았고, 이어서 {rest} 부분까지 넓혀 가며 정
리했…
반환 f"{opener.rstrip('.')} {first} 흐름으로 전반적인 내용을 익혔어요."
```

_phase_unit_narratives

`study_note_record_organization/scripts/build_output.py:1287`

```
def _phase_unit_narratives(item: CatalogItem) -> list[tuple[str, str]]
```

장·강·일차가 많을 때 앞·중·뒤 구간만 구어체로 요약.

- 인자: item: CatalogItem
- 반환: list[tuple[str, str]]

유사코드

```
n ← len(item.sources)
만약 n < 4
  반환 []
sources ← item.sources
만약 n <= 6
  phases ← [(0, 0, '처음'), (n // 2, n // 2, '중간'), (n - 1, n - 1, '마지막')]
아니면
  third ← max(1, n // 3)
  phases ← [(0, third - 1, '앞부분'), (third, 2 * third - 1, '중반'), (2 * third, n - 1…
blocks ← []
반복 (start, end, label) ← phases
  seg ← sources[start:end + 1]
  u_from ← seg[0].unit
  u_to ← seg[-1].unit
  unit_label ← u_from if u_from == u_to else f'{u_from}~{u_to}'
  만약 label in ('처음', '앞부분')
    para ← f'{unit_label} 구간에서는 {item.summary}의 기초를 다지며 개념과 환경 설정에 집중했어요.'
  아니면
    만약 label in ('마지막', '후반')
      para ← f'{unit_label} 구간에서는 지금까지 배운 내용을 연결·정리하고 실전·응용 쪽으로 마무리했
어요.'
    아니면
      para ← f'{unit_label} 구간에서는 예제·실습을 따라 하며 손에 익히는 단계를 밟았어요.'
  blocks.append((unit_label, para))
반환 blocks
```

build_study_narrative

`study_note_record_organization/scripts/build_output.py:1433`

```
def build_study_narrative(item: CatalogItem) -> dict[str, Any]
```

과목 전체를 서술형 구어체로 요약 (원문 나열 없음).

- 인자: item: CatalogItem
- 반환: dict[str, Any]

유사코드

```
unit_hint ← ''
만약 len(item.sources) >= 3
  units ← [s.unit for s in item.sources if s.unit != '전체']
  만약 units
    unit_hint ← f' ({units[0]}부터 {units[-1]}까지)'
overview ← f'「{item.title}」을(를) 공부하면서{unit_hint} 총 {len(item.sources)}개의 노트를 남겼어요.…
main ← STUDY_NARRATIVES.get(item.slug)
credit ← STUDY_CREDITS.get(item.slug)
만약 credit and main and (not main.startswith(credit))
  main ← f'{credit} {main}'
만약 not main
  main ← f'이 과목에서는 {item.summary}을(를) 목표로 {len(item.sources)}번에 걸쳐 내용을 쌓아 갔어요. 노…
reflection ← f'돌이켜 보면 {item.title}에서 {item.summary}와 관련된 내용을 차근차근 쌓아 갔어요. 원본 문장을 그대로…
반환 {'overview': overview, 'main': main, 'reflection': reflection, 'units':…
```

summarize_bullets

`study_note_record_organization/scripts/build_output.py:1473`

```
def summarize_bullets(text: str, max_items: int=10) -> list[str]
```

글머리표, 번호 목록, 긴 줄을 중복 없이 최대 개수만큼 모아 요약 불릿으로 만든다.

- 인자: text: str, max_items: int = 10
- 반환: list[str]

유사코드

```
lines ← [ln.strip() for ln in text.splitlines() if ln.strip()]
bullets ← []
반복 ln ← lines
  만약 len(ln) < 4
    다음 반복
  만약 ln.startswith(('-', '*', '•'))
    bullets.append(ln.lstrip('-*• ').strip())
  아니면
    만약 re.match('^\\d+[\\.\\)]', ln)
      bullets.append(re.sub('^\\d+[\\.\\)]\\s*', '', ln))
    아니면
      만약 len(ln) > 15 and (not ln.startswith('='))
        bullets.append(ln)
seen ← set()
out ← []
반복 b ← bullets
  key ← b[:60]
  만약 key not in seen
    seen.add(key)
    out.append(b)
  만약 len(out) >= max_items
    반복 종료
반환 out
```

make_overview

`study_note_record_organization/scripts/build_output.py:1498`

```
def make_overview(item: CatalogItem, bullets: list[str]) -> str
```

학습이면 노트 수와 키워드로, 과제이면 수행 기록과 원본 파일 수로 한눈에 보기 문장을 만든다.

- 인자: item: CatalogItem, bullets: list[str]
- 반환: str

유사코드

```
만약 item.type == 'assignment'
  반환 f'{item.title} 과목의 총괄/최종 과제를 수행하며 과제 요구사항에 맞춰 답안·코드·문서를 작성했습니다. 원본 자료는 …
만약 bullets
  topics ← '·'.join((b[:20] for b in bullets[:3]))
  반환 f'「{item.title}」 학습을 진행하며 {len(item.sources)}개 단위의 노트를 남겼습니다. 주요 키워드: {…
반환 f'「{item.title}」 관련 학습 노트를 정리했습니다. 원본 메모가 짧거나 키워드 수준일 수 있습니다.'
```

render_windows_programming1_markdown

`study_note_record_organization/scripts/build_output.py:1518`

```
def render_windows_programming1_markdown(item: CatalogItem) -> str
```

실행 파일 제출 과제 — 텍스트 원문 없이 프로그램 화면으로 설명.

- 인자: item: CatalogItem
- 반환: str

유사코드

```
lines ← ['---', f'id: {item.id}', f'type: {item.type}', f'category: {item.categ…
반환 '\n'.join(lines)
```

render_markdown

`study_note_record_organization/scripts/build_output.py:1573`

```
def render_markdown(item: CatalogItem) -> str
```

카탈로그 항목 하나의 프론트매터와 본문 마크다운을 만든다. 윈도우 프로그래밍1은 별도 형식을 쓴다.

- 인자: item: CatalogItem
- 반환: str

유사코드

```
만약 item.slug == 'windows-programming-1'
  반환 render_windows_programming1_markdown(item)
all_text ← '\n\n'.join((s.text for s in item.sources if s.text))
bullets ← summarize_bullets(all_text)
study_narr ← build_study_narrative(item) if item.type == 'study' else None
overview ← study_narr['overview'] if study_narr else make_overview(item, bullets)
fm_summary ← item.summary
progress_units ← len(item.sources)
fm_sources ← [{'path': s.path, 'unit': s.unit} for s in item.sources]
lines ← ['---', f'id: {item.id}', f'type: {item.type}', f'category: {item.categ…
반복 s ← fm_sources
  path ← mask_assignment_privacy(s['path']) if item.type == 'assignment' else s[…
  lines.append(f' - path: {path}')
  lines.append(f''' unit: "{s['unit']}"''')
lines.extend(['created_from: study_note_record_organization', '---', ''…
lines.append(f'# {item.title}\n')
lines.append('## 한눈에 보기\n')
lines.append(overview + '\n')
만약 item.type == 'study' and study_narr
  lines.append('## 무엇을 배웠는지\n')
  lines.append(study_narr['main'] + '\n\n')
  lines.append(study_narr['reflection'] + '\n\n')
  만약 study_narr['units']
    lines.append('## 단계별로 살보면\n')
    반복 (unit, para) ← study_narr['units']
      lines.append(f'### {unit}\n')
      lines.append(para + '\n\n')
아니면
  만약 item.type != 'assignment'
    lines.append('## 핵심 요약\n')
    만약 bullets
      반복 b ← bullets
        lines.append(f'- {b}')
    아니면
      lines.append('- 원본 메모가 키워드 수준이거나 추출 텍스트가 비어 있습니다.')
    lines.append('')
만약 item.type == 'assignment'
  lines.append('## 내용 요약\n')
  summary_lines ← build_assignment_content_summary(item, all_text)
  반복 para ← summary_lines
…
```

main

`study_note_record_organization/scripts/build_output.py:1657`

```
def main() -> None
```

학습·과제 요약을 output에 쓰고 catalog.json과 index.md를 갱신한 뒤 정적 사이트 빌드를 호출한다.

- 인자: 없음
- 반환: None

유사코드

```
STUDY_OUT.mkdir(parents=True, exist_ok=True)
ASSIGN_OUT.mkdir(parents=True, exist_ok=True)
EXTRACTED.mkdir(parents=True, exist_ok=True)
study_items ← build_study_catalog()
assign_items ← build_assignment_catalog()
all_items ← study_items + assign_items
print(f'Study: {len(study_items)}, Assignment: {len(assign_items)}, Tot…
반복 item ← all_items
  item.load_sources()
expected_study ← {f'{it.slug}.md' for it in study_items}
expected_assign ← {f'{it.slug}.md' for it in assign_items}
반복 (d, expected) ← ((STUDY_OUT, expected_study), (ASSIGN_OUT, expected_assign))
  반복 old ← d.glob('*.md')
    만약 old.name not in expected
      old.unlink()
catalog_data ← []
반복 item ← all_items
  md ← render_markdown(item)
  out_dir ← STUDY_OUT if item.type == 'study' else ASSIGN_OUT
  (out_dir / f'{item.slug}.md').write_text(md, encoding='utf-8')
  catalog_data.append({'id': item.id, 'type': item.type, 'category': item…
(OUT / 'catalog.json').write_text(json.dumps({'items': catalog_data, 'c…
idx ← ['# 학습·과제 기록 인덱스\n', f'총 **{len(all_items)}**개 항목 (학습 {len(study_items)…
idx.append('## 학습 (study)\n')
반복 it ← study_items
  idx.append(f'- [{it.title}](study/{it.slug}.md) — {it.summary}')
idx.append('\n## 과제 (assignment)\n')
반복 it ← assign_items
  idx.append(f'- [{it.title}](assignment/{it.slug}.md) — {it.summary}')
(OUT / 'index.md').write_text('\n'.join(idx), encoding='utf-8')
print('Done.')
시도
  모듈을 불러온다
  모듈을 불러온다
  script ← ROOT / 'scripts' / 'build_static_site.py'
  만약 script.is_file()
    subprocess.run([sys.executable, str(script)], check=False)
실패 OSError
```

study_note_record_organization/scripts/build_static_site.py

요약 마크다운과 원본 텍스트로 대시보드용 data.js, 소스 JSON, 히어로 목록을 만든다.

mask_secrets

`study_note_record_organization/scripts/build_static_site.py:51`

```
def mask_secrets(text: str, mask: str=KEY_MASK) -> str
```

본문에서 API 키·토큰 패턴을 찾아 고정 마스크 문자열로 바꾼다.

- 인자: text: str, mask: str = KEY_MASK
- 반환: str

유사코드

```
만약 not text
  반환 text
반복 pat ← _SECRET_PATTERNS
  text ← pat.sub(mask, text)
반복 pat ← _KEY_ASSIGNMENT_PATTERNS
  text ← pat.sub(f'\\g<1>{mask}\\g<3>', text)
반환 text
```

is_assignment_path

`study_note_record_organization/scripts/build_static_site.py:67`

```
def is_assignment_path(rel_path: str) -> bool
```

경로에 과제 폴더 이름이 있으면 참을 돌려준다.

- 인자: rel_path: str
- 반환: bool

유사코드

```
반환 _ASSIGNMENT_DIR in rel_path.replace('\\', '/')
```

mask_assignment_privacy

`study_note_record_organization/scripts/build_static_site.py:72`

```
def mask_assignment_privacy(text: str) -> str
```

과제 본문의 지도교수 이름, 학번, 파일명 속 학번 숫자를 가린다.

- 인자: text: str
- 반환: str

유사코드

```
만약 not text
  반환 text
text ← re.sub('(지도교수\\s*[:：]\\s*)\\S+(?:\\s*교수님)?', f'\\1{PROFESSOR_MASK}', te…
text ← re.sub('((?:학\\s*번|수\\s*강\\s*번\\s*호)\\s*[:：]\\s*)\\d+', f'\\1{STUDENT_I…
text ← re.sub('\\(\\d{7,8}\\s+', f'({STUDENT_ID_MASK} ', text)
text ← re.sub('\\(\\d{7,8}\\)', f'({STUDENT_ID_MASK})', text)
text ← re.sub('_\\d{7,8}_', f'_{STUDENT_ID_MASK}_', text)
반환 text
```

parse_frontmatter

`study_note_record_organization/scripts/build_static_site.py:92`

```
def parse_frontmatter(raw: str) -> tuple[dict, str]
```

마크다운 앞의 YAML 프론트매터를 딕셔너리와 본문으로 나눈다. 태그 목록과 sources도 읽는다.

- 인자: raw: str
- 반환: tuple[dict, str]

유사코드

```
match ← re.match('^---\\r?\\n([\\s\\S]*?)\\r?\\n---\\r?\\n([\\s\\S]*)$', raw)
만약 not match
  반환 ({}, raw)
(yaml, body) ← (match.group(1), match.group(2))
meta ← {}
current_key ← None
in_sources ← False
source_item ← None
반복 line ← yaml.split('\n')
  만약 line.strip().startswith('sources:')
    in_sources ← True
    meta['sources'] ← []
    다음 반복
  만약 in_sources
    만약 re.match('^\\s+-\\s+path:', line)
      source_item ← {'path': line.split('path:', 1)[1].strip()}
      meta['sources'].append(source_item)
    아니면
      만약 source_item and re.match('^\\s+unit:', line)
        source_item['unit'] ← line.split('unit:', 1)[1].strip().strip('"\'')
      아니면
        만약 line.strip() and (not line.startswith(' '))
          in_sources ← False
  만약 not in_sources
    m ← re.match('^(\\w+):\\s*(.*)$', line)
    만약 m
      current_key ← m.group(1)
      val ← m.group(2).strip().strip('"\'')
      meta[current_key] ← val
    아니면
      만약 current_key == 'tags' and re.match('^\\s+-\\s+', line)
        meta.setdefault('tags', [])
        meta['tags'].append(line.split('-', 1)[1].strip())
만약 isinstance(meta.get('tags'), str) and meta['tags'].startswith('[')
  시도
    meta['tags'] ← json.loads(meta['tags'].replace("'", '"'))
  실패 json.JSONDecodeError
    meta['tags'] ← []
반환 (meta, body)
```

md_to_html

`study_note_record_organization/scripts/build_static_site.py:135`

```
def md_to_html(body: str) -> str
```

요약 마크다운 본문을 HTML로 바꾼다. markdown 패키지가 없으면 제목·목록·문단만 직접 변환한다.

- 인자: body: str
- 반환: str

유사코드

```
시도
  모듈을 불러온다
  반환 markdown.markdown(body, extensions=['extra', 'nl2br', 'sane_lists'], ou…
실패 ImportError
  html ← []
  반복 block ← re.split('\\n\\n+', body.strip())
    block ← block.strip()
    만약 not block
      다음 반복
    만약 block.startswith('### ')
      html.append(f'<h3>{_esc(block[4:])}</h3>')
    아니면
      만약 block.startswith('## ')
        html.append(f'<h2>{_esc(block[3:])}</h2>')
      아니면
        만약 block.startswith('# ')
          html.append(f'<h1>{_esc(block[2:])}</h1>')
        아니면
          만약 block.startswith('- ')
            items ← ''.join((f'<li>{_inline_md(line[2:])}</li>' for line in block.split('\n…
            html.append(f'<ul>{items}</ul>')
          아니면
            html.append(f"<p>{_inline_md(block.replace(chr(10), ' '))}</p>")
  반환 '\n'.join(html)
```

_esc

`study_note_record_organization/scripts/build_static_site.py:169`

```
def _esc(s: str) -> str
```

HTML에 넣을 문자열의 &, <, >, 따옴표를 이스케이프한다.

- 인자: s: str
- 반환: str

유사코드

```
반환 s.replace('&', '&amp;').replace('<', '&lt;').replace('>', '&gt;').repla…
```

_inline_md

`study_note_record_organization/scripts/build_static_site.py:179`

```
def _inline_md(s: str) -> str
```

한 줄 안의 굵게, 코드, 사이트 내부 링크를 HTML로 바꾼다.

- 인자: s: str
- 반환: str

유사코드

```
s ← _esc(s)
s ← re.sub('\\*\\*(.+?)\\*\\*', '<strong>\\1</strong>', s)
s ← re.sub('`([^`]+)`', '<code>\\1</code>', s)
s ← re.sub('\\[([^\\]]+)\\]\\((/[^)]+)\\)', '<a href="\\2">\\1</a>', s)
반환 s
```

normalize_source_key

`study_note_record_organization/scripts/build_static_site.py:192`

```
def normalize_source_key(key: str) -> str
```

마크다운 링크의 퍼센트 인코딩을 풀어 디스크의 원본 경로와 같게 만든다.

- 인자: key: str
- 반환: str

유사코드

```
k ← key.replace('\\', '/').lstrip('/')
prev ← None
동안 prev != k
  prev ← k
  k ← unquote(k)
반환 k
```

rewrite_source_links

`study_note_record_organization/scripts/build_static_site.py:202`

```
def rewrite_source_links(html: str, pending: set[str]) -> str
```

본문 HTML의 /data, /_extracted 링크를 원본 보기용 data-source 앵커로 바꾸고, 필요한 원본 키를 모은다.

- 인자: html: str, pending: set[str]
- 반환: str

유사코드

```
repl 을 정의한다
반환 _SOURCE_HREF_RE.sub(repl, html)
```

rewrite_source_links.repl

`study_note_record_organization/scripts/build_static_site.py:204`

```
def repl(m: re.Match) -> str
```

링크 하나에서 원본 키를 꺼내 pending에 넣고, source-link 클래스가 있는 앵커로 바꾼다.

- 인자: m: re.Match
- 반환: str

유사코드

```
href ← m.group(1)
key ← normalize_source_key(href)
pending.add(key)
attrs ← m.group(2) or ''
만약 'class=' in attrs
  attrs ← re.sub('class="([^"]*)"', lambda cm: f'class="{cm.group(1)} source-link…
아니면
  attrs ← attrs 연산 ' class="source-link"'
반환 f'<a href="#" data-source="{key}"{attrs}>'
```

resolve_source_path

`study_note_record_organization/scripts/build_static_site.py:224`

```
def resolve_source_path(key: str) -> Path | None
```

원본 키를 output/_extracted 또는 저장소 data 아래의 실제 파일 경로로 바꾼다.

- 인자: key: str
- 반환: Path | None

유사코드

```
key ← normalize_source_key(key)
만약 key.startswith('_extracted/')
  반환 OUT / key
만약 key.startswith('data/')
  반환 ROOT / key
반환 None
```

read_text_file

`study_note_record_organization/scripts/build_static_site.py:234`

```
def read_text_file(path: Path) -> str
```

utf-8, utf-8-sig, cp949 순으로 읽고, 실패하면 깨진 글자를 대체해 utf-8로 읽는다.

- 인자: path: Path
- 반환: str

유사코드

```
반복 enc ← ('utf-8', 'utf-8-sig', 'cp949')
  시도
    반환 path.read_text(encoding=enc)
  실패 (UnicodeDecodeError, OSError)
    다음 반복
반환 path.read_text(encoding='utf-8', errors='replace')
```

read_source_text

`study_note_record_organization/scripts/build_static_site.py:244`

```
def read_source_text(key: str) -> str | None
```

원본 키의 파일을 읽어 비밀 값을 가리고, 과제면 개인정보도 가린 뒤 돌려준다. 파일이 없으면 None이다.

- 인자: key: str
- 반환: str | None

유사코드

```
path ← resolve_source_path(key)
만약 not path or not path.is_file()
  반환 None
시도
  text ← read_text_file(path)
실패 OSError
  반환 None
text ← mask_secrets(text)
만약 is_assignment_path(key)
  text ← mask_assignment_privacy(text)
반환 text
```

collect_hero_images

`study_note_record_organization/scripts/build_static_site.py:259`

```
def collect_hero_images() -> list[str]
```

data/image의 이미지 파일을 정렬해 대시보드에서 쓰는 상대 경로 목록으로 만든다.

- 인자: 없음
- 반환: list[str]

유사코드

```
만약 not HERO_IMAGE_DIR.is_dir()
  반환 []
paths ← []
반복 path ← sorted(HERO_IMAGE_DIR.iterdir(), key=lambda p: p.name.lower())
  만약 not path.is_file()
    다음 반복
  만약 path.suffix.lower() not in HERO_IMAGE_EXTS
    다음 반복
  paths.append(f'../data/image/{path.name}')
반환 paths
```

write_hero_images_js

`study_note_record_organization/scripts/build_static_site.py:273`

```
def write_hero_images_js(js_dir: Path) -> int
```

히어로 이미지 경로를 window.HERO_IMAGES 대입문으로 hero-images.js에 쓰고 개수를 돌려준다.

- 인자: js_dir: Path
- 반환: int

유사코드

```
images ← collect_hero_images()
content ← 'window.HERO_IMAGES = ' + json.dumps(images, ensure_ascii=False) + ';\n'
(js_dir / 'hero-images.js').write_text(content, encoding='utf-8')
반환 len(images)
```

source_chunk_filename

study_note_record_organization/scripts/build_static_site.py:281

```
def source_chunk_filename(key: str, text: str) -> str
```

원본 키와 본문 해시로 sources 폴더에 저장할 JSON 파일 이름을 만든다.

- 인자: key: str, text: str
- 반환: str

유사코드

```
key_digest ← hashlib.sha256(key.encode('utf-8')).hexdigest()[:8]
content_digest ← hashlib.sha256(text.encode('utf-8')).hexdigest()[:8]
반환 f'{key_digest}-{content_digest}.json'
```

source_build_id

study_note_record_organization/scripts/build_static_site.py:288

```
def source_build_id(source_map: dict[str, str]) -> str
```

원본 지도 전체를 해시해 캐시 무효화에 쓰는 짧은 빌드 아이디를 만든다.

- 인자: source_map: dict[str, str]
- 반환: str

유사코드

```
payload ← '|'.join((f'{k}:{v}' for k, v in sorted(source_map.items())))
반환 hashlib.sha256(payload.encode('utf-8')).hexdigest()[:12]
```

stamp_index_scripts

`study_note_record_organization/scripts/build_static_site.py:294`

```
def stamp_index_scripts(build_id: str) -> None
```

대시보드 index.html의 data.js, app.js, styles.css 주소에 빌드 아이디 쿼리를 붙인다. 원본 지도가 그대로여도 화면 스크립트와 스타일이 바뀌면 브라우저 캐시를 갱신한다.

- 인자: build_id: str
- 반환: None

유사코드

```
index_path ← SITE / 'index.html'
ui_parts ← [build_id]
반복 rel ← ('js/app.js', 'css/styles.css')
  path ← SITE / rel
  만약 path.is_file()
    ui_parts.append(hashlib.sha256(path.read_bytes()).hexdigest()[:8])
token ← hashlib.sha256('|'.join(ui_parts).encode('utf-8')).hexdigest()[:12]
html ← index_path.read_text(encoding='utf-8')
html ← re.sub('src="js/(data|app)\\.js(?:\\?[^"]*)?"', lambda m: f'src="js/{m.…
html ← re.sub('href="css/styles\\.css(?:\\?[^"]*)?"', f'href="css/styles.css?v…
index_path.write_text(html, encoding='utf-8')
```

assert_under_github_limit

`study_note_record_organization/scripts/build_static_site.py:320`

```
def assert_under_github_limit(path: Path, label: str) -> None
```

파일 크기가 GitHub 한 파일 한도를 넘으면 빌드를 중단한다.

- 인자: path: Path, label: str
- 반환: None

유사코드

```
size ← path.stat().st_size
만약 size > GITHUB_MAX_BYTES
  mb ← size / (1024 * 1024)
  중단 SystemExit(f'{label} ({path}) is {mb:.1f} MB — exceeds GitHub 100 MB fi…
```

collect_catalog

`study_note_record_organization/scripts/build_static_site.py:330`

```
def collect_catalog(catalog_data: dict) -> list[dict]
```

catalog.json 항목에서 카드에 필요한 id, 종류, 분류, 제목, 슬러그, 요약, 태그를 고른다.

- 인자: catalog_data: dict
- 반환: list[dict]

유사코드

```
items ← []
반복 it ← catalog_data.get('items', [])
  items.append({'id': it['id'], 'type': it['type'], 'category': it.get('c…
반환 items
```

build_site

`study_note_record_organization/scripts/build_static_site.py:348`

```
def build_site() -> None
```

요약 HTML, 원본 JSON, data.js, 히어로 목록을 dashboard에 쓴다. catalog.json이 없으면 중단한다.

- 인자: 없음
- 반환: None

유사코드

```
만약 not CATALOG_PATH.is_file()
  중단 SystemExit('catalog.json 없음. 먼저 python scripts/build_output.py 실행')
catalog_data ← json.loads(CATALOG_PATH.read_text(encoding='utf-8'))
catalog ← collect_catalog(catalog_data)
articles ← {}
pending_sources ← set()
반복 item ← catalog_data.get('items', [])
  md_path ← ROOT / item['output']
  만약 not md_path.is_file()
    다음 반복
  raw ← md_path.read_text(encoding='utf-8')
  (_, body) ← parse_frontmatter(raw)
  html ← md_to_html(mask_secrets(body))
  html ← rewrite_source_links(html, pending_sources)
  articles[item['slug']] ← html
source_map ← {}
sources_dir ← SITE / 'js' / 'sources'
만약 sources_dir.is_dir()
  shutil.rmtree(sources_dir)
sources_dir.mkdir(parents=True, exist_ok=True)
written_sources ← 0
sources_total_bytes ← 0
반복 key ← sorted(pending_sources)
  text ← read_source_text(key)
  만약 text is None
    다음 반복
  chunk_name ← source_chunk_filename(key, text)
  chunk_path ← sources_dir / chunk_name
  chunk_payload ← {'key': key, 'text': text}
  chunk_path.write_text(json.dumps(chunk_payload, ensure_ascii=False), en…
  assert_under_github_limit(chunk_path, 'Source chunk')
  source_map[key] ← chunk_name
  written_sources ← written_sources 연산 1
  sources_total_bytes ← sources_total_bytes 연산 chunk_path.stat().st_size
build_id ← source_build_id(source_map)
site_data ← {'catalog': catalog, 'articles': articles, 'sourceMap': source_map, 'so…
js_dir ← SITE / 'js'
css_dir ← SITE / 'css'
js_dir.mkdir(parents=True, exist_ok=True)
css_dir.mkdir(parents=True, exist_ok=True)
…
```

main

`study_note_record_organization/scripts/build_static_site.py:422`

```
def main() -> None
```

build_site를 실행하고, 만든 대시보드를 브라우저에서 열 수 있다고 안내한다.

- 인자: 없음
- 반환: None

유사코드

```
build_site()
print('Done. Open dashboard/index.html in a browser (or use a local sta…
```

docs/build_portfolio_book.py

포트폴리오 북 마크다운 챕터들을 통합하여 A4 규격의 한글 PDF 단행본으로 빌드한다. 마크다운 표, 인용구, 코드 블록, 인라인 링크 및 Mermaid 다이어그램을 파싱하여 Microsoft Edge 또는 Google Chrome 헤드리스 엔진으로 고품질 인쇄용 PDF를 생성한다.

inline

`docs/build_portfolio_book.py:34`

```
def inline(text: str) -> str
```

설명이 없습니다.

- 인자: text: str
- 반환: str

유사코드

```
escaped ← html.escape(text)
escaped ← INLINE_CODE.sub(lambda m: f'<code>{m.group(1)}</code>', escaped)
escaped ← BOLD.sub(lambda m: f'<strong>{m.group(1)}</strong>', escaped)
escaped ← LINK.sub(lambda m: f'<a href="{m.group(2)}">{m.group(1)}</a>', escaped)
반환 escaped
```

find_browser

`docs/build_portfolio_book.py:42`

```
def find_browser() -> Path | None
```

설명이 없습니다.

- 인자: 없음
- 반환: Path | None

유사코드

```
local ← os.environ.get('LOCALAPPDATA', '')
candidates ← [Path('C:\\Program Files (x86)\\Microsoft\\Edge\\Application\\msedge.ex…
반복 candidate ← candidates
  만약 candidate.is_file()
    반환 candidate
반환 None
```

diagram_page

`docs/build_portfolio_book.py:56`

```
def diagram_page(source: str) -> str
```

설명이 없습니다.

- 인자: source: str
- 반환: str

유사코드

```
escaped ← html.escape(source)
반환 f'<!DOCTYPE html>\n<html lang="ko">\n<head>\n<meta charset="utf-8" />\n…
```

crop_whitespace

`docs/build_portfolio_book.py:79`

```
def crop_whitespace(path: Path) -> None
```

설명이 없습니다.

- 인자: path: Path
- 반환: None

유사코드

```
image ← Image.open(path).convert('RGB')
background ← Image.new('RGB', image.size, (255, 255, 255))
box ← ImageChops.difference(image, background).getbbox()
만약 box is None
  중단 SystemExit(f'차트가 비어 있습니다: {path.name}')
pad ← 16
left ← max(box[0] - pad, 0)
top ← max(box[1] - pad, 0)
right ← min(box[2] + pad, image.width)
bottom ← min(box[3] + pad, image.height)
cropped ← image.crop((left, top, right, bottom))
만약 cropped.width < 40 or cropped.height < 40
  중단 SystemExit(f'차트가 너무 작습니다: {path.name}')
cropped.save(path)
```

render_diagram

`docs/build_portfolio_book.py:96`

```
def render_diagram(source: str) -> str
```

설명이 없습니다.

- 인자: source: str
- 반환: str

유사코드

```
digest ← hashlib.sha256(source.encode('utf-8')).hexdigest()[:12]
USED_DIAGRAMS.add(digest)
DIAGRAMS.mkdir(parents=True, exist_ok=True)
png ← DIAGRAMS / f'{digest}.png'
만약 not png.is_file()
  browser ← find_browser()
  만약 browser is None
    escaped ← html.escape(source.strip())
    반환 f'<pre class="code diagram-fallback">{escaped}</pre>'
  html_path ← DIAGRAMS / f'{digest}.html'
  html_path.write_text(diagram_page(source), encoding='utf-8')
  profile ← Path(tempfile.mkdtemp(prefix='book-chart-'))
  command ← [str(browser), '--headless=new', '--disable-gpu', '--no-first-run', '--…
  시도
    subprocess.run(command, check=True, timeout=60)
    deadline ← time.time() + 8
    동안 not png.is_file() and time.time() < deadline
      time.sleep(0.3)
    crop_whitespace(png)
  실패 Exception
    print(f'[경고] 차트 렌더링 실패 ({digest}): {e}')
    escaped ← html.escape(source.strip())
    반환 f'<pre class="code diagram-fallback">{escaped}</pre>'
rel ← png.relative_to(ROOT).as_posix()
반환 f'<figure class="chart"><img src="{rel}" alt="다이어그램" /></figure>'
```

parse_table

`docs/build_portfolio_book.py:140`

```
def parse_table(lines: list[str]) -> str
```

설명이 없습니다.

- 인자: lines: list[str]
- 반환: str

유사코드

```
rows ← []
반복 line ← lines
  raw_cells ← [c.strip() for c in line.strip().strip('|').split('|')]
  rows.append(raw_cells)
만약 not rows
  반환 ''
header ← rows[0]
data_rows ← rows[1:]
만약 data_rows and all((re.match('^:?-+:?$', cell) for cell in data_rows[0]))
  data_rows ← data_rows[1:]
html_parts ← ['<div class="table-wrap"><table><thead><tr>']
반복 cell ← header
  html_parts.append(f'<th>{inline(cell)}</th>')
html_parts.append('</tr></thead><tbody>')
반복 r ← data_rows
  html_parts.append('<tr>')
  반복 cell ← r
    html_parts.append(f'<td>{inline(cell)}</td>')
  html_parts.append('</tr>')
html_parts.append('</tbody></table></div>')
반환 ''.join(html_parts)
```

markdown_to_html

`docs/build_portfolio_book.py:166`

```
def markdown_to_html(text: str) -> str
```

설명이 없습니다.

- 인자: text: str
- 반환: str

유사코드

```
lines ← text.splitlines()
parts ← []
index ← 0
first_h1 ← True
동안 index < len(lines)
  line ← lines[index]
  만약 line.startswith('```')
    language ← line[3:].strip()
    body ← []
    index ← index 연산 1
    동안 index < len(lines) and (not lines[index].startswith('```'))
      body.append(lines[index])
      index ← index 연산 1
    index ← index 연산 1
    source ← '\n'.join(body).strip() + '\n'
    만약 language == 'mermaid'
      parts.append(render_diagram(source))
    아니면
      parts.append(f'<pre class="code"><code>{html.escape(source)}</code></pr…
    다음 반복
  stripped ← line.strip()
  만약 not stripped
    index ← index 연산 1
    다음 반복
  만약 stripped in ('---', '***', '___')
    parts.append('<hr class="divider" />')
    index ← index 연산 1
    다음 반복
  만약 line.startswith('#')
    level ← len(line) - len(line.lstrip('#'))
    level ← min(max(level, 1), 4)
    title ← line[level:].strip()
    cls ← ''
    만약 level == 1
      만약 first_h1
        cls ← ' class="book-title"'
        first_h1 ← False
      아니면
        cls ← ' class="chapter-title"'
    parts.append(f'<h{level}{cls}>{inline(title)}</h{level}>')
…
```

combined_markdown

`docs/build_portfolio_book.py:281`

```
def combined_markdown() -> str
```

설명이 없습니다.

- 인자: 없음
- 반환: str

유사코드

```
files ← sorted(CHAPTERS.glob('*.md'))
만약 not files
  중단 SystemExit(f'포트폴리오 북 챕터 마크다운이 없습니다: {CHAPTERS}')
chunks ← [path.read_text(encoding='utf-8').strip() for path in files]
반환 '\n\n'.join(chunks) + '\n'
```

render_html

`docs/build_portfolio_book.py:497`

```
def render_html(body: str) -> str
```

설명이 없습니다.

- 인자: body: str
- 반환: str

유사코드

```
반환 '\n'.join(['<!DOCTYPE html>', '<html lang="ko">', '<head>', '<meta char…
```

write_pdf

`docs/build_portfolio_book.py:517`

```
def write_pdf(html_path: Path, pdf_path: Path) -> None
```

설명이 없습니다.

- 인자: html_path: Path, pdf_path: Path
- 반환: None

유사코드

```
browser ← find_browser()
만약 browser is None
  중단 SystemExit('PDF로 인쇄할 Edge 또는 Chrome을 찾지 못했습니다.')
만약 pdf_path.exists()
  pdf_path.unlink()
profile ← Path(tempfile.mkdtemp(prefix='portfolio-book-pdf-'))
command ← [str(browser), '--headless=new', '--disable-gpu', '--no-first-run', '--…
시도
  subprocess.run(command, check=True, timeout=120)
만약 not pdf_path.is_file() or pdf_path.stat().st_size < 1000
  중단 SystemExit(f'PDF가 만들어지지 않았습니다: {pdf_path}')
```

main

`docs/build_portfolio_book.py:542`

```
def main() -> None
```

설명이 없습니다.

- 인자: 없음
- 반환: None

유사코드

```
print('[1/4] 포트폴리오 북 마크다운 챕터 병합 중...')
markdown ← combined_markdown()
MD_PATH.write_text(markdown, encoding='utf-8')
print(f' -> 통합 마크다운 저장: {MD_PATH}')
print('[2/4] 마크다운 -> HTML 변환 및 다이어그램 렌더링 중...')
html_content ← render_html(markdown_to_html(markdown))
HTML_PATH.write_text(html_content, encoding='utf-8')
print(f' -> HTML 저장: {HTML_PATH}')
print('[3/4] 헤드리스 브라우저 A4 PDF 변환 중...')
write_pdf(HTML_PATH, PDF_PATH)
print(f' -> PDF 완성: {PDF_PATH} ({PDF_PATH.stat().st_size // 1024} KB)')
print('[4/4] 빌드 완료! 성공적으로 포트폴리오 북이 생성되었습니다.')
```

생성 산출물

다음 파일은 사람이 함수를 적어 둔 소스가 아니다. build_static_site.py를 다시 실행하면 data.js, hero-images.js, sources JSON의 본문이 바뀌므로 그 안에 설명 주석을 넣지 않았다.

- study_note_record_organization/dashboard/js/data.js — 카드 목록, 요약 HTML, 원본 파일 지도를 window.SITE_DATA로 넣는다.
- study_note_record_organization/dashboard/js/hero-images.js — data/image 경로를 window.HERO_IMAGES 배열로 넣는다.
- study_note_record_organization/dashboard/js/sources/*.json — 원본 텍스트 170개. 키와 본문만 담는다.

학습 원본 파일 목록

data 폴더의 노트, 과제 문서, 이미지다. 함수가 없어 파일마다 한 줄만 적는다. 코드 파일은 넣지 않았고, 원문 전문도 넣지 않았다. 학번과 이름은 가렸다.

.docx

- study_note_record_organization/data/바로바로 바이브 코딩 with 커서 AI 독서노트.docx — 독서 노트